

Jonas Lucas Durão
168893

Projeto e Implementação de um Processador Baseado na Arquitetura MIPS em FPGA

São José dos Campos - Brasil

Junho de 2025

Jonas Lucas Durão
168893

Projeto e Implementação de um Processador Baseado na Arquitetura MIPS em FPGA

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Junho de 2025

Resumo

Ao longo da disciplina de Arquitetura e Organização de Computadores, foram abordados conceitos teóricos fundamentais sobre o funcionamento e a construção de processadores, suas características de projeto e os principais componentes internos. Entre os componentes estudados, destacam-se a memória, a unidade lógica e aritmética (ULA), o conjunto de instruções, entre outros elementos essenciais para a arquitetura de um processador. Esses conhecimentos teóricos foram aplicados de forma prática no desenvolvimento do processador. Neste projeto, utilizando uma linguagem de descrição de hardware, será implementado um processador funcional baseado na arquitetura MIPS. O processador incluirá os módulos básicos dessa arquitetura, como banco de registradores, unidade lógica e aritmética (ULA), memória principal e de instruções, além das interconexões entre esses componentes. Adicionalmente, serão incorporados módulos extras para garantir a realização de operações com pilha, que serão úteis em futuras disciplinas. O processador desenvolvido será implementado na plataforma FPGA, permitindo a execução de programas e a simulação de seu funcionamento real.

Palavras-chaves: Arquitetura e Organização de Computadores. MIPS. RISC. Kit FPGA.

Lista de ilustrações

Figura 1 – Arquitetura de Von Neumann e de Harvard	12
Figura 2 – Endereçamento Direto	16
Figura 3 – Endereçamento Direto	17
Figura 4 – Endereçamento Direto	18
Figura 5 – FPGA DE2-115 Cyclone IV	20
Figura 6 – Diagrama Datapath Geral do Processador	28
Figura 7 – Diagrama do Datapath da instrução <i>add</i>	30
Figura 8 – Diagrama do Datapath da instrução <i>addi</i>	31
Figura 9 – Diagrama do Datapath da instrução <i>lwr</i>	32
Figura 10 – Diagrama do Datapath da instrução <i>beq</i>	33
Figura 11 – Waveform para as entradas e saídas da ULA para todas as operações da Tabela 5	66
Figura 12 – Forma de onda da simulação para o módulo Banco de Registradores.	67
Figura 13 – Forma de onda da simulação para o módulo MemoriaDados.	68
Figura 14 – Forma de onda da simulação para o módulo MemoriaInstrucoes.	69
Figura 15 – Forma de onda da simulação para o módulo PC.	70
Figura 16 – Forma de onda da simulação do módulo Processador integrado.	72
Figura 17 – Teste Fibonacci entrada 1	74
Figura 18 – Teste Fibonacci entrada 10	74
Figura 19 – Teste Fibonacci entrada 16	75
Figura 20 – Teste JR e JAL entrada 1	76
Figura 21 – Teste JR e JAL entrada 17	77
Figura 22 – Teste JR e JAL entrada 512	77

Lista de tabelas

Tabela 1 – OPCODE e Mnemônicos das Instruções	23
Tabela 2 – Instrução x Operação	24
Tabela 3 – Formatos e Instruções Correspondentes	26
Tabela 4 – Sinais de Controle e suas Funções	29
Tabela 5 – Códigos de Operação da ULA (ALUop)	34
Tabela 6 – Endereços dos Registradores Específicos	37
Tabela 7 – Resultados dos Casos de Teste da ULA	66
Tabela 8 – Resultados dos Casos de Teste do Banco de Registradores	68
Tabela 9 – Resultados dos Casos de Teste da Memória de Dados	69
Tabela 10 – Resultados dos Casos de Teste da Memória de Instruções	70
Tabela 11 – Resultados dos Casos de Teste do PC por Borda de Clock	71
Tabela 12 – Resultados dos Testes do Processador por Etapa da Simulação	72

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	9
2.1	Geral	9
2.2	Específicos	9
3	FUNDAMENTAÇÃO TEÓRICA	10
3.1	Arquitetura e Organização de Computadores	10
3.2	Arquiteturas RISC e CISC	10
3.3	Arquitetura RISC	10
3.3.1	Arquitetura CISC	11
3.4	Arquitetura de Von Neumann e Harvard	12
3.5	Fluxo dos Dados	13
3.6	Processador MIPS	13
3.6.1	Características Principais	14
3.6.2	Formatos de Instrução	14
3.6.3	Aplicações e Impacto	15
3.6.4	Evolução e Estado Atual	15
3.7	Modos de Endereçamento	15
3.7.1	Direto	15
3.7.2	Indireto de Registrador	16
3.7.3	Base-Deslocamento por Registrador	17
3.8	Linguagem de Descrição de Hardware: Verilog	18
3.9	<i>Field Programmable Gate Array (FPGA)</i>	19
4	DESENVOLVIMENTO	21
4.1	Características Gerais do Processador	21
4.2	Conjunto de Instruções	22
4.2.1	Formato de Instruções	25
4.3	Modos de Endereçamento	26
4.3.1	Endereçamento Direto	26
4.3.2	Endereçamento Indireto por Registrador	27
4.3.3	Endereçamento Base-Deslocamento por Registrador	27
4.4	Caminho de Dados	27
4.4.1	Instrução <i>add</i> - Tipo R - Formato OPR	29
4.4.2	Instrução <i>addi</i> - Tipo I - Formato OPI	30

4.4.3	Instrução <i>lwr</i> - Tipo R - Formato OPI	31
4.4.4	Instrução <i>beq</i> - Tipo J - Formato B	32
4.5	Implementação do Projeto	33
4.5.1	Unidade Lógica e Aritmética (ULA)	34
4.5.2	Banco de Registradores	36
4.5.3	Memória de Dados (RAM)	39
4.5.4	Memória de Instrução (ROM)	40
4.5.5	Registrador Contador de Programa (PC)	40
4.5.6	Multiplexadores	42
4.5.7	Somador	42
4.5.8	Extensor de Bits	43
4.5.9	Unidade de Controle	44
4.5.10	Unidade de Controle da Pilha	51
4.5.11	Divisor de Frequência	52
4.5.12	Filtro Debouncer	53
4.5.13	Conversor Binário para BCD	55
4.5.14	Decodificador BCD para 7 Segmentos	56
4.5.15	Módulo de Entrada/Saída (I/O)	57
4.5.16	Integração Final dos Módulos	58
5	RESULTADOS OBTIDOS E DISCUSSÕES	65
5.1	Formas de Onda	65
5.1.1	Teste da Unidade Lógica e Aritmética	65
5.1.2	Teste do Banco de Registradores	67
5.1.3	Teste da Memória RAM	68
5.1.4	Teste da Memória ROM	69
5.1.5	Teste do PC	70
5.1.6	Teste Integração Banco de Registradores e ULA	71
5.2	Teste no <i>kit</i> FPGA	73
5.2.1	Teste Fibonacci	73
5.2.2	Teste JR e JAL	75
6	CONSIDERAÇÕES FINAIS	78
	REFERÊNCIAS	79

1 Introdução

O processador constitui o núcleo de um sistema computacional, sendo responsável pela coordenação e execução das instruções que sustentam o funcionamento do computador. Dentre suas estruturas fundamentais, destacam-se a Unidade Lógica e Aritmética (ULA), os registradores e a memória principal, que viabilizam operações essenciais como processamento, armazenamento, transferência e controle de dados (STALLINGS, 2010). Esses componentes são cruciais para a execução de códigos e algoritmos, possibilitando a realização de tarefas computacionais complexas com eficiência e precisão. A implementação de um processador é um processo complexo, que é dividido em diversas etapas.

Essas etapas incluem o projeto do processador, como a escolha da arquitetura a ser utilizada, a definição do número de registradores, o tipo de fluxo de dados e a quantidade de módulos a serem implementados. Cada uma dessas características impacta diretamente a funcionalidade do processador. Sem um processador eficiente e bem projetado, seria impossível realizar tarefas simples, como navegar na internet, ou operações mais complexas, como a simulação de modelos científicos ou a execução de softwares avançados. A evolução dos processadores ao longo dos anos reflete diretamente o avanço da tecnologia computacional, impactando desde os sistemas de computação pessoal até os supercomputadores, que realizam cálculos de alta performance em áreas como inteligência artificial, pesquisa biomédica e física de partículas.

Nos primeiros computadores, os processadores eram limitados em termos de velocidade, capacidade de processamento e consumo de energia. No entanto, ao longo das décadas, foram feitos avanços significativos em miniaturização, capacidade de integração e aumento da velocidade de processamento. As primeiras gerações de processadores eram baseadas em arquiteturas simples, com unidades de controle e execução básicas. No entanto, o desenvolvimento de novos conceitos, como a arquitetura de *pipelining*, paralelismo e técnicas de múltiplos núcleos, possibilitou uma expansão exponencial da capacidade computacional.

Além disso, o aprimoramento das tecnologias de fabricação, como o uso de transistores mais rápidos e menores, permitiu a redução do tamanho físico dos processadores, ao mesmo tempo em que aumentava sua performance. As inovações no design dos processadores também trouxeram melhorias no consumo de energia e na dissociação térmica, dois fatores cruciais para a operação de dispositivos modernos, desde *smartphones* até servidores em data centers.

Dessa forma, estudar arquitetura e organização de computadores é fundamental para compreender não apenas como os processadores funcionam, mas também como os

sistemas computacionais como um todo são projetados. O conhecimento da arquitetura de processadores permite que engenheiros de hardware e software otimizem a interação entre o processador e os demais componentes do sistema, como a memória e os periféricos, garantindo uma execução mais eficiente de programas e algoritmos. Em um mundo cada vez mais dependente de sistemas computacionais, o entendimento profundo de como esses sistemas operam e evoluem é uma habilidade essencial para quem deseja contribuir para o avanço da ciência da computação e suas diversas aplicações.

2 Objetivos

2.1 Geral

O objetivo geral deste trabalho é projetar e implementar um processador funcional, a partir dos fundamentos teóricos adquiridos na disciplina de Arquitetura e Organização de Computadores. O processador será desenvolvido para execução em um kit FPGA, contemplando a definição da arquitetura e a implementação dos principais módulos que compõem sua estrutura interna.

2.2 Específicos

Os objetivos específicos deste projeto são:

1. Estudar a arquitetura MIPS e seu modelo de projeto, com o intuito de embasar a implementação de um processador com características compatíveis com essa arquitetura;
2. Definir o conjunto de instruções a ser implementado, bem como os respectivos formatos a serem reconhecidos e decodificados pelo processador;
3. Determinar a quantidade de bits da *opcode* e dos operandos, de modo a assegurar compatibilidade com o conjunto de instruções estabelecido;
4. Estabelecer a quantidade e a finalidade dos registradores, incluindo registradores de uso geral, específicos e campos imediatos;
5. Especificar os modos de endereçamento a serem utilizados nas instruções de acesso à memória;
6. Projetar um caminho de dados SISD (Single Instruction, Single Data) baseado em arquitetura monociclo, capaz de suportar todas as instruções definidas para o processador.
7. Implementar os principais módulos que compõem um processador, tais como: Unidade Lógica e Aritmética (ULA), Unidade de Controle, Unidade de Controle da Pilha, Memória de Instruções e Dados, além do módulo de Entrada e Saída (I/O);
8. Desenvolver o processador projetado utilizando uma linguagem de descrição de hardware e verificar seu funcionamento em um kit FPGA.

3 Fundamentação Teórica

3.1 Arquitetura e Organização de Computadores

Para descrever computadores, é necessário estabelecer a distinção entre a arquitetura e a organização de um computador. A arquitetura de computador refere-se aos atributos de um sistema que são visíveis ao programador, ou seja, aqueles que impactam diretamente na execução lógica de um programa ([STALLINGS, 2010](#)). Já a organização de computador refere-se às unidades operacionais e suas intercomunicações que implementam as especificações definidas pela arquitetura ([STALLINGS, 2010](#)).

Como um exemplo, se um computador terá uma instrução de soma seria uma questão de arquitetura, já uma questão de organização seria se eu for ter um componente no processador para realizar operações de soma.

3.2 Arquiteturas RISC e CISC

Podemos definir diferentes arquiteturas de processadores, sendo que duas delas formam a base para a maior parte dos processadores utilizados atualmente: as arquiteturas RISC (Reduced Instruction Set Computer) e CISC (Complex Instruction Set Computer). Ambas apresentam características distintas, que as tornam adequadas para diferentes necessidades e aplicações na computação. Essas arquiteturas definem como o conjunto de instruções será estruturado e processado pelo processador, influenciando diretamente sua performance, eficiência e complexidade.

3.3 Arquitetura RISC

A arquitetura **RISC** caracteriza-se pelo uso de um conjunto reduzido de instruções simples e rápidas. O princípio central do RISC é otimizar a execução das instruções, simplificando-as para aumentar a eficiência e a velocidade de execução. Algumas das principais propriedades que definem o RISC, conforme descrito em ([STALLINGS, 2010](#)), são:

- Execução de uma instrução por ciclo de clock.
- Realização de operações entre registradores (exceto nas operações de *load* e *store*, que acessam a memória).
- Utilização de modos de endereçamento simples.

- Estrutura de formatos de instruções simples e uniformes.

Usualmente arquiteturas RISC utilizam um único ciclo de clock para realizar suas instruções. Porém, outra vantagem importante do RISC é sua implementação baseada em **pipeline**, o que significa que as instruções são executadas em no máximo 5 ciclos de clock, em uma sequência precisa: ciclo de busca da instrução, decodificação da instrução e busca dos registradores, execução da operação e cálculo do endereço efetivo. Para instruções que acessam a memória, o processo inclui um ciclo de acesso à memória seguido por um ciclo de *write-back* (HENNESSY; PATTERSON, 2012).

Essas características tornam a arquitetura RISC uma escolha ideal para otimizar a execução de operações simples e rápidas, contribuindo para maior eficiência. Dessa maneira, é uma escolha ideal para sistemas que exigem alta performance com eficiência energética, como servidores, dispositivos móveis e sistemas embarcados.

3.3.1 Arquitetura CISC

A arquitetura **CISC** é caracterizada por um conjunto de instruções mais complexo e abrangente, permitindo a execução de operações em múltiplos ciclos de clock. O surgimento dessa nova abordagem foi motivado pela busca por compiladores mais simples e melhor desempenho, impulsionada pela adoção de linguagens de alto nível pelos programadores (STALLINGS, 2010). Essa arquitetura visa reduzir a quantidade de instruções necessárias para realizar uma tarefa, oferecendo comandos mais poderosos e abrangentes. Algumas das principais características da arquitetura CISC, definidas em (HEATH, 2014), incluem:

- As instruções CISC levam mais de 1 ciclo de clock para serem executadas.
- Possui instruções complexas.
- As instruções têm tamanho variável.
- A ênfase está no hardware.
- Usa um número muito reduzido de registradores.
- Não é utilizada como uma máquina de load e store.
- Não é muito eficiente em termos de velocidade de compilação comparado ao RISC.
- Não utiliza *pipelining*.
- Para armazenar instruções complexas, são usados transistores.

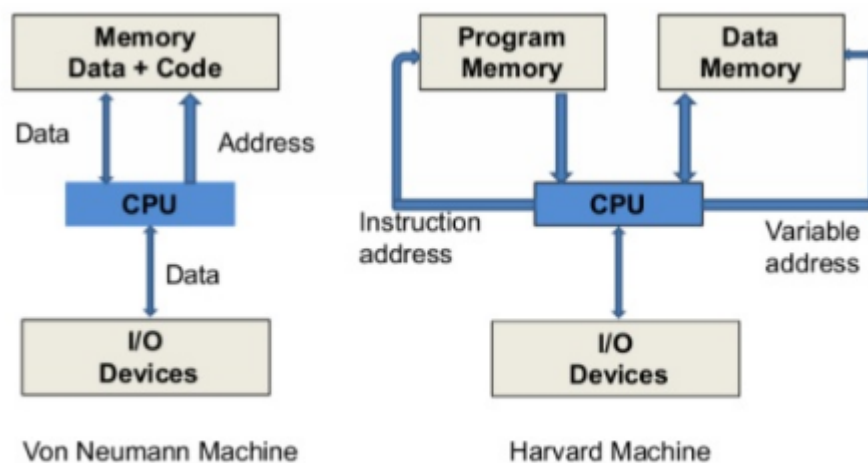
Com essas características, o CISC é ideal para facilitar a tarefa do programador de compiladores, melhorar a eficiência da execução (pois sequências complexas de operações podem ser implementadas no microcódigo) e fornecer suporte para linguagens de programação de alto nível ainda mais complexas e sofisticadas (STALLINGS, 2010).

3.4 Arquitetura de Von Neumann e Harvard

A arquitetura de Von Neumann, proposta por John Von Neumann em 1945, serve como a base para a maioria dos processadores modernos. Ela caracteriza-se pelo uso de um único caminho de dados para instruções e dados, o que significa que ambos são armazenados na mesma memória e acessados por um único barramento (HENNESSY; PATTERSON, 2012). Essa configuração pode levar à formação de um gargalo de desempenho, pois o processador precisa aguardar a recuperação da instrução para então acessar o dado, o que limita a eficiência do sistema.

Em contraste, a arquitetura Harvard adota memórias separadas para dados e instruções. Isso possibilita que o processador realize operações simultâneas de leitura de instrução e acesso a dados, eliminando o gargalo presente na arquitetura de Von Neumann. Essa separação entre os caminhos de dados resulta em maior eficiência e velocidade, tornando a arquitetura Harvard mais adequada para sistemas que exigem desempenho mais alto, como microcontroladores e sistemas embarcados.

Figura 1 – Arquitetura de Von Neumann e de Harvard



Fonte: (EDITORIAIS DA TMLT, 2021)

Na Figura 1, é possível perceber exatamente como ocorre a separação das memórias de instrução na arquitetura Harvard, e também o compartilhamento do barramento no

caso de Von Neumann.

3.5 Fluxo dos Dados

O conceito de fluxo de dados foi abordado por Flynn (1966), que propôs uma classificação de arquiteturas de computadores com base no número de fluxos de instrução e dados simultâneos. As categorias propostas são: SISD (Single Instruction Stream, Single Data Stream), SIMD (Single Instruction Stream, Multiple Data Streams), MISD (Multiple Instruction Streams, Single Data Stream) e MIMD (Multiple Instruction Streams, Multiple Data Streams) (HENNESSY; PATTERSON, 2012).

A arquitetura SISD é o modelo tradicional de um único processador, onde um fluxo de instrução e um de dados são processados sequencialmente. Já a SIMD permite que múltiplos processadores executem a mesma instrução em diferentes dados, explorando o paralelismo em nível de dados, amplamente utilizado em GPUs e sistemas de processamento vetorial. Em contraste, a MIMD permite que cada processador tenha seu próprio fluxo de instruções e dados, buscando paralelismo em nível de tarefa. Essa arquitetura é mais flexível e comum em supercomputadores e clusters de computadores.

Por fim, a categoria MISD completa a taxonomia de Flynn, mas não tem aplicação prática, uma vez que não há implementações comerciais conhecidas dessa arquitetura.

3.6 Processador MIPS

O processador MIPS (Microprocessor without Interlocked Pipeline Stages – Microprocessador Sem Estágios Intertravados de Pipeline) é uma família de processadores criada pela empresa MIPS Computer System em 1985. Seu modelo de arquitetura é amplamente utilizado como referência no estudo de arquitetura e organização de computadores, tanto por sua grande popularidade quanto por seu design didático, o que o torna particularmente acessível para iniciantes na área (PATTERSON; HENNESSY, 2006).

A arquitetura MIPS é um exemplo clássico da arquitetura RISC, projetada inicialmente por John Hennessy e sua equipe na Universidade de Stanford, com o objetivo de explorar os benefícios de um conjunto de instruções reduzido e de execução eficiente por meio de pipeline ((ORG.), 1999).

Todas as instruções MIPS possuem tamanho fixo de 32 bits, organizadas em formatos regulares e previsíveis que facilitam a decodificação e a implementação em hardware. Além disso, adota o modelo load/store, característico das arquiteturas RISC, no qual apenas instruções específicas acessam a memória, enquanto operações aritméticas e lógicas são realizadas entre registradores.

Outro ponto de destaque é a implementação de pipeline com cinco estágios — busca, decodificação, execução, acesso à memória e escrita no registrador — permitindo a execução simultânea de instruções e aumentando o desempenho geral do processador ((ORG.), 1999). Por sua clareza estrutural e eficiência, a arquitetura MIPS continua sendo amplamente adotada tanto em aplicações práticas quanto no ensino acadêmico.

3.6.1 Características Principais

- **Conjunto de Instruções:** O MIPS adota um conjunto de instruções reduzido e regular, facilitando a decodificação e execução das operações. Cada instrução possui um formato fixo de 32 bits, o que simplifica o processo de busca e decodificação no pipeline do processador.
- **Arquitetura Load/Store:** Todas as operações aritméticas e lógicas são realizadas entre registradores. O acesso à memória é restrito a instruções específicas de carga (load) e armazenamento (store), o que contribui para a previsibilidade e eficiência do pipeline.
- **Uso de Pipeline:** O MIPS implementa um pipeline de cinco estágios (busca de instrução, decodificação, execução, acesso à memória e escrita no registrador), permitindo a execução simultânea de múltiplas instruções em diferentes estágios, aumentando o *throughput* (número de instruções que consegue executar por segundo) do processador.
- **Registradores de Propósito Geral:** Possui 32 registradores de propósito geral, cada um com 32 bits de largura. Essa abundância de registradores reduz a necessidade de acessos frequentes à memória, melhorando o desempenho geral do sistema.
- **Modularidade e Extensibilidade:** A arquitetura MIPS é modular, permitindo a inclusão de coprocessadores, como unidades de ponto flutuante, para estender suas capacidades conforme as necessidades específicas de aplicações.

3.6.2 Formatos de Instrução

As instruções MIPS são categorizadas em três formatos principais:

- **Formato R (*Register*):** Utilizado para operações que envolvem apenas registradores. Inclui campos para especificar registradores de origem e destino, além de um campo de função que determina a operação específica a ser realizada.
- **Formato I (*Immediate*):** Empregado em instruções que envolvem um valor imediato (constante). Contém campos para registradores de origem e destino, além de um campo para o valor imediato.

- **Formato J (*Jump*)**: Destinado a instruções de desvio, como saltos incondicionais. Inclui um campo de OP-`CODE` e um campo de endereço de destino.

3.6.3 Aplicações e Impacto

A arquitetura MIPS teve ampla adoção em diversas áreas, desde sistemas embarcados até consoles de videogame e roteadores. Sua simplicidade e eficiência a tornaram uma escolha popular para aplicações que exigem alto desempenho com baixo consumo de energia. Além disso, o MIPS serviu como base para o ensino de arquitetura de computadores em instituições acadêmicas, devido à sua clareza conceitual e estrutura organizada.

3.6.4 Evolução e Estado Atual

Ao longo dos anos, a arquitetura MIPS passou por várias atualizações, incluindo versões de 64 bits e extensões para suportar operações de ponto flutuante e instruções SIMD (Single Instruction, Multiple Data). Recentemente, a empresa responsável pela arquitetura MIPS anunciou uma mudança estratégica, focando no desenvolvimento de chips voltados para aplicações em robótica e inteligência artificial, utilizando a arquitetura RISC-V como base para futuras implementações ([NELLIS, 2025](#)).

3.7 Modos de Endereçamento

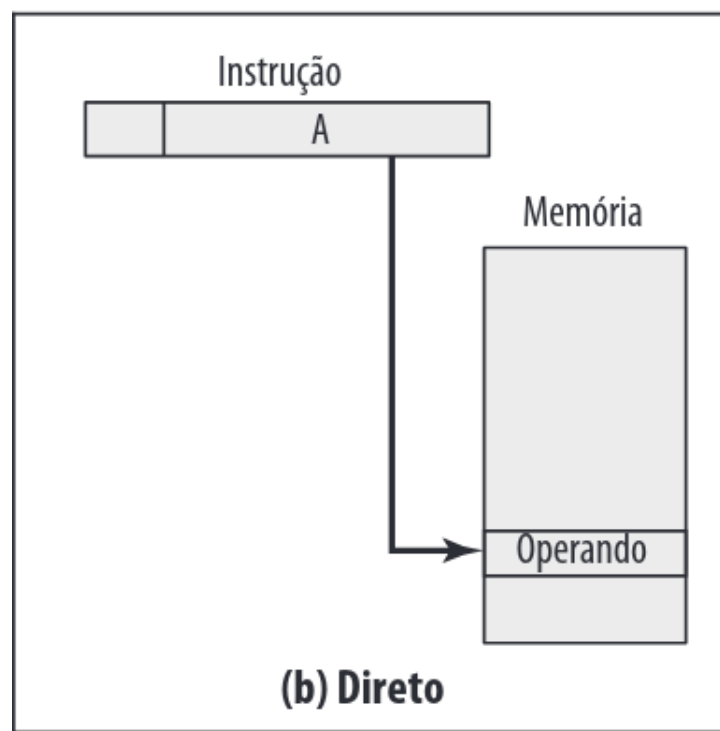
Os modos de endereçamento são técnicas utilizadas para calcular o endereço efetivo de um operando na memória durante a execução de uma instrução. A forma como esse endereço é calculado varia de acordo com o modo de endereçamento especificado na instrução. Para alcançar esse objetivo, diferentes abordagens foram desenvolvidas, envolvendo um equilíbrio entre a flexibilidade de endereçamento, o intervalo de endereços, o número de referências de memória e a complexidade do cálculo do endereço. Nesta seção, exploraremos as técnicas de endereçamento mais comuns ([STALLINGS, 2010](#)).

Embora exista uma vasta gama de modos — como o imediato, indexado, relativo ao PC e de pilha —, este projeto concentra-se na implementação de um subconjunto essencial que oferece um balanço ideal entre simplicidade e poder computacional. A fundamentação teórica e a implementação serão focadas em três modos principais, cujas características serão detalhadas nas seções a seguir.

3.7.1 Direto

No modo de endereçamento direto, o endereço efetivo é fornecido diretamente na instrução. Ou seja, a instrução contém o endereço da memória onde o operando está localizado.

Figura 2 – Endereçamento Direto



Fonte: (STALLINGS, 2010)

- **Endereço Efetivo:** O endereço é o valor diretamente especificado na instrução.
- **Operando:** O operando é armazenado na memória no endereço especificado pela instrução. Não há processamento adicional para localizar o operando, pois o endereço é dado diretamente.

Exemplo de formato de instrução:

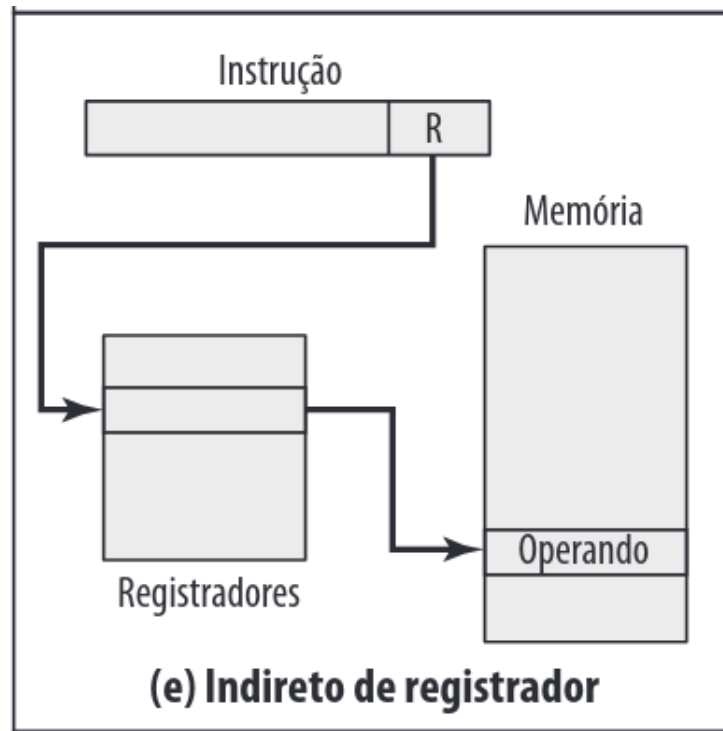
```
LOAD R1, 1000
```

Neste caso, o endereço efetivo é 1000, e o valor armazenado nesse endereço será transferido para o registrador R1.

3.7.2 Indireto de Registrador

No modo de endereçamento indireto de registrador, o endereço efetivo é determinado pelo conteúdo de um registrador. O valor armazenado nesse registrador é utilizado como o endereço para acessar a memória.

Figura 3 – Endereçamento Direto



Fonte: (STALLINGS, 2010)

- **Endereço Efetivo:** O endereço é o valor armazenado no registrador especificado na instrução.
- **Operando:** O operando é armazenado no endereço de memória que é encontrado no registrador.

Exemplo de formato de instrução:

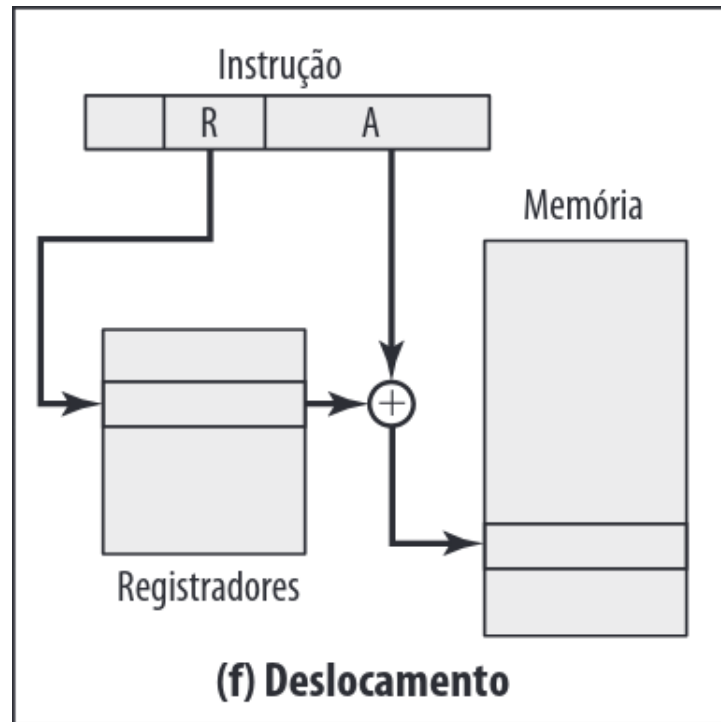
```
LOAD R1, R2
```

Neste caso, o endereço efetivo será o valor armazenado no registrador R2, e o valor na memória nesse endereço será copiado para o registrador R1.

3.7.3 Base-Deslocamento por Registrador

No modo de endereçamento base-deslocamento por registrador, o endereço efetivo é calculado somando o valor armazenado em um registrador base com um deslocamento constante que é especificado na instrução.

Figura 4 – Endereçamento Direto



Fonte: (STALLINGS, 2010)

- **Endereço Efetivo:** O endereço é calculado pela soma do conteúdo de um registrador base com um deslocamento especificado.
- **Operando:** O operando é localizado na memória no endereço resultante dessa soma.

Exemplo de formato de instrução:

```
LOAD R1, R2, 10
```

Neste caso, o endereço efetivo é calculado somando o valor armazenado em R2 com o deslocamento de 10. O valor encontrado no endereço resultante será copiado para R1.

3.8 Linguagem de Descrição de Hardware: Verilog

O Verilog é uma das principais linguagens de descrição de hardware (HDL) amplamente utilizada no desenvolvimento de sistemas digitais, especialmente em circuitos integrados e sistemas em FPGA. Assim como as linguagens de programação de maior nível, como C, o Verilog segue uma sintaxe que permite descrever o comportamento e a estrutura de circuitos digitais de forma textual e abstrata, facilitando o design, simulação e síntese de sistemas.

A principal vantagem do Verilog em comparação aos modelos esquemáticos tradicionais, como os usados em diagramas de blocos, é a expressividade e flexibilidade. Ele permite que o programador defina o comportamento dos circuitos em diferentes níveis de abstração, desde descrições de alto nível (como a lógica funcional) até a descrição de portas e componentes de hardware específicos. Isso possibilita a criação de sistemas complexos de forma mais eficiente e com menos propensão a erros.

3.9 *Field Programmable Gate Array (FPGA)*

O FPGA DE2-115 Cyclone IV é um dispositivo de *Field-Programmable Gate Array* (FPGA) desenvolvido pela Altera (adquirida pela Intel em 2015). Esse tipo de hardware é notável por sua capacidade de ser programado e reprogramado várias vezes, o que o torna ideal para aplicações educacionais, prototipagem e desenvolvimento de sistemas digitais. Ao contrário dos dispositivos convencionais, que são programados durante o processo de manufatura, os FPGAs oferecem a flexibilidade de se adaptar a diferentes funcionalidades após sua fabricação.

Esse dispositivo é amplamente utilizado em contextos educacionais, como no desenvolvimento de processadores e sistemas digitais, devido à sua versatilidade e ao ambiente de aprendizado que proporciona. O FPGA DE2-115 utiliza a arquitetura Cyclone IV, uma das mais populares para dispositivos de baixo custo e alto desempenho, oferecendo um bom equilíbrio entre capacidade de processamento e consumo de energia.

Entre as características principais do FPGA DE2-115 Cyclone IV, destacam-se os displays LED de sete segmentos e os LEDs que permitem a visualização de dados de saída. Além disso, o dispositivo possui componentes de entrada como chaves e botões, facilitando a interação com o sistema desenvolvido, como pode ser visualizado na [Figura 5](#). Essas características tornam o FPGA uma plataforma ideal para experimentação e aprendizado prático na área de sistemas digitais e design de hardware.

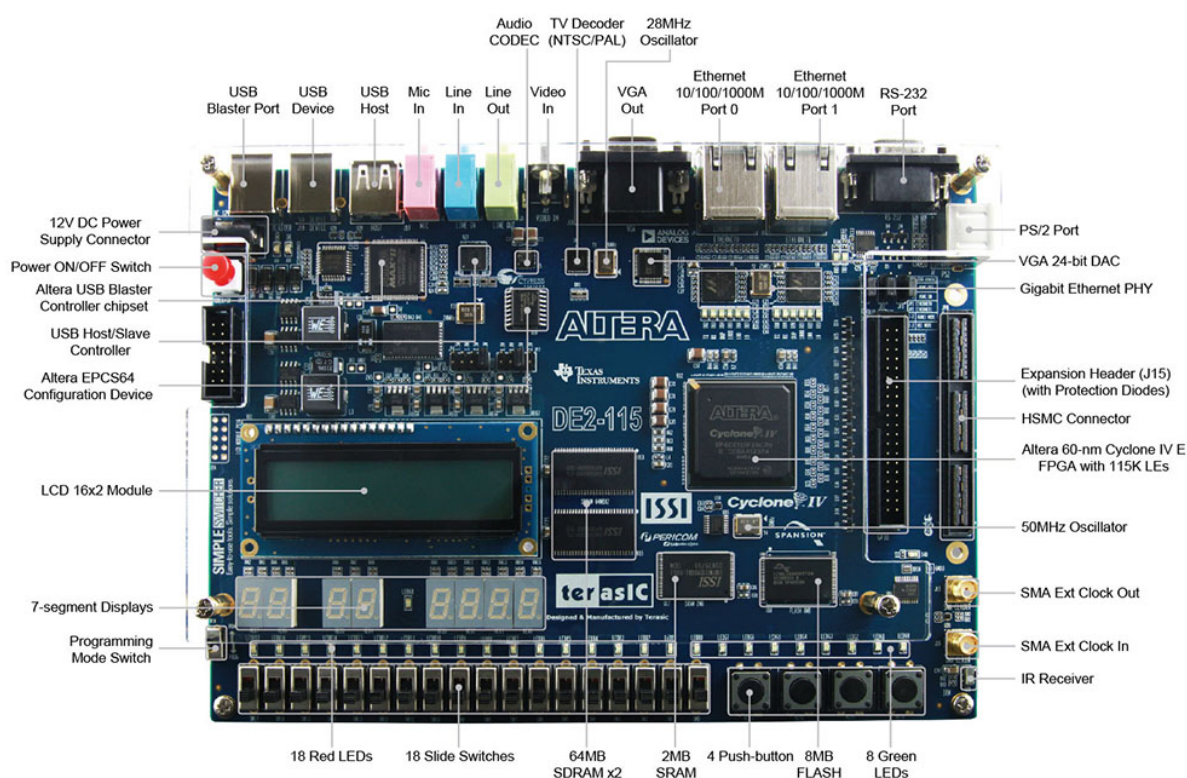


Figura 5 – FPGA DE2-115 Cyclone IV

4 Desenvolvimento

4.1 Características Gerais do Processador

O processador a ser desenvolvido apresenta as seguintes características gerais que determinam seu comportamento e estrutura de funcionamento:

- **Arquitetura Base:** O processador proposto segue a arquitetura **MIPS**, que, por ser baseada no **RISC**, adota instruções simples e de execução eficiente, facilitando tanto o desenvolvimento quanto a implementação da lógica do processador.
- **Tamanho das Instruções:** Cada instrução possui um tamanho fixo de **32 bits**, organizados em campos distintos responsáveis por definir a operação a ser executada e seus respectivos operandos. Entre os principais campos, destacam-se:
 - **OPCODE:** Campo de 6 bits que identifica a operação a ser realizada.
 - **Registradores:** Campo de 6 bits destinado à identificação dos registradores envolvidos na instrução.
 - **Imediatos:** Campos de tamanho variável (14, 20 ou 26 bits), utilizados para representar valores imediatos ou endereços, conforme o tipo de instrução. Essa variação permite maior flexibilidade e otimização no uso de diferentes operações.
- **Número de Instruções:** O conjunto de instruções do processador é composto por **36 operações distintas**, cobrindo funções aritméticas, lógicas, de controle e de acesso à memória. Essa variedade proporciona versatilidade no desenvolvimento de algoritmos e aplicações.
- **Registradores:** O processador conta com **64 registradores**, dos quais **3 possuem funções específicas**. O registrador `$rp` é utilizado para controle de pilha, enquanto o registrador `$ra` é reservado para armazenar o valor do contador de programa (PC) durante a execução da instrução `jal` (Jump and Link). Essa organização garante ao usuário uma combinação entre flexibilidade e suporte a operações de controle.
- **Formatos de Instrução:** Serão adotados **6 formatos distintos de instrução**, projetados para otimizar diferentes tipos de operação. Cada formato foi desenvolvido com base em padrões de uso comuns, como múltiplos registradores, manipulação de valores imediatos, controle de fluxo e operações de acesso à memória.
- **Fluxo de Dados:** A arquitetura do processador segue o modelo **Harvard**, com separação física entre as memórias de instrução e de dados. Essa separação permite

o acesso simultâneo a ambos os tipos de informação, aumentando a eficiência do sistema. A execução das instruções ocorrerá em modo **Monociclo**, ou seja, cada instrução será concluída em um único ciclo de clock. Além disso, o processador adota o modelo de processamento **SISD**, executando uma instrução por vez de forma sequencial.

- **Modos de Endereçamento para Acesso à Memória:** O processador oferecerá suporte a três modos distintos de endereçamento para operações de leitura e escrita em memória:
 1. **Endereçamento Direto:** O endereço do operando é especificado diretamente na instrução.
 2. **Endereçamento Indireto por Registrador:** O endereço efetivo é fornecido pelo conteúdo de um registrador.
 3. **Endereçamento por Base com Deslocamento:** O endereço efetivo é obtido pela soma de um registrador base com um valor de deslocamento definido na instrução.

Essas características tornam o processador eficiente, flexível e adequado para uma ampla gama de aplicações, desde simples operações aritméticas até acessos complexos à memória. A escolha da arquitetura Harvard, com seu caminho de dados monociclo, aliada ao modelo MIPS e aos modos de endereçamento variados, visa garantir um desempenho robusto e escalável.

4.2 Conjunto de Instruções

Para a definição das instruções foi pensado nas mais utilizadas na produção de algoritmos em Assembly MIPS e as propostas pelo professor, o que possibilita um uso prolongado do processador. Dessa forma, a [Tabela 1](#) mostra as instruções que serão implementadas no processador.

Tabela 1 – OP CODE e Mnemônicos das Instruções

Opcode	Instrução	Opcode	Instrução
000000	add	010010	ble
000001	sub	010011	move
000010	mult	010100	li
000011	div	010101	lw
000100	AND	010110	sw
000101	OR	010111	lwr
000110	NOT	011000	swr
000111	addi	011001	lwd
001000	subi	011010	swd
001001	multi	011011	j
001010	ANDI	011100	jr
001011	ORI	011101	jal
001100	sr	011110	PUSH
001101	sl	011111	POP
001110	bge	100000	IN
001111	beq	100001	OUT
010000	bgt	100010	NOP
010001	blt	100011	HLT

Tabela 2 – Instrução x Operação

Instrução	Operação Realizada
Operações Aritméticas:	
add [RD, RS, RT]	$RD \leftarrow RS + RT$
addi [RD, RS, IM14]	$RD \leftarrow RS + IM14$
sub [RD, RS, RT]	$RD \leftarrow RS - RT$
subi [RD, RS, IM14]	$RD \leftarrow RS - IM14$
mult [RD, RS, RT]	$RD \leftarrow RS * RT$ (32 bits máximo)
multi [RD, RS, IM14]	$RD \leftarrow RS * IM14$ (32 bits máximo)
div [RD, RS, RT]	$RD \leftarrow RS / RT$
Operações Lógicas:	
AND [RD, RS, RT]	$RD \leftarrow RS \text{ and } RT$
ANDI [RD, RS, IM14]	$RD \leftarrow RS \text{ and } IM14$
OR [RD, RS, RT]	$RD \leftarrow RS \text{ or } RT$
ORI [RD, RS, IM14]	$RD \leftarrow RS \text{ or } IM14$
NOT [RD, RS]	$RD \leftarrow \text{NOT } (RS)$
Deslocamento:	
sr [RD, RS]	$RD \leftarrow \text{DESL}(RS) \text{ p/ direita uma vez } (RS / 2)$
sl [RD, RS]	$RD \leftarrow \text{DESL}(RS) \text{ p/ esquerda uma vez } (RS * 2)$
Transferência de Dados:	
move [RD, RS]	$RD \leftarrow RS$
lw [RD, ENDEREÇO]	$RD \leftarrow \text{RAM_dados}[\text{ENDEREÇO}]$
sw [RS, ENDEREÇO]	$\text{RAM_dados}[\text{ENDEREÇO}] \leftarrow RS$
lwr [RD, RS]	$RD \leftarrow \text{RAM_dados}[\text{ENDEREÇO}[RS]]$
swr [RS, RT]	$\text{RAM_dados}[\text{ENDEREÇO}[RT]] \leftarrow RS$
lwd [RD, RS, IM14]	$RD \leftarrow \text{RAM_dados}[\text{ENDEREÇO}[RS] + IM14]$
swd [RS, RT, IM14]	$\text{RAM_dados}[\text{ENDEREÇO}[RT] + IM14] \leftarrow RS$
li [RD, IM20]	$RD \leftarrow IM20$
PUSH [RS]	$\$rp = \$rp + 1 \text{ e } \text{PILHA}[\$rp] \leftarrow RS$
POP [RD]	$RD \leftarrow \text{PILHA}[\$rp] \text{ e } \$rp = \$rp - 1$
Salto incondicional:	
j [IM26]	$PC \leftarrow IM26$
jr [RS]	$PC \leftarrow RS$
jal [IM26]	$\$ra = PC + 1 \text{ e } PC \leftarrow IM26$
Salto condicional:	
beq [RS, RT, IM14]	$PC \leftarrow PC + IM14, \text{ se } RS == RT$
bgt [RS, RT, IM14]	$PC \leftarrow PC + IM14, \text{ se } RS > RT$
blt [RS, RT, IM14]	$PC \leftarrow PC + IM14, \text{ se } RS < RT$
bge [RS, RT, IM14]	$PC \leftarrow PC + IM14, \text{ se } RS \geq RT$
ble [RS, RT, IM14]	$PC \leftarrow PC + IM14, \text{ se } RS \leq RT$
Controle:	
NOP []	Nenhuma operação
HLT []	Parar Processamento
Entrada e Saída:	
IN [RD]	$RD \leftarrow \text{Slide-Switches}$
OUT [RS]	$\text{Display7Seg} \leftarrow \text{Valor_BCD}(RD)$

A [Tabela 2](#) ilustra as operações que o processador realizará para cada instrução a ser implementada. Dessa forma, é possível visualizar como o processador reagirá a cada instrução durante a execução do código.

4.2.1 Formato de Instruções

Montar os formatos das instruções é uma etapa crucial no design de um processador, pois define como as instruções serão interpretadas e executadas no *DataPath*, sendo que cada operação exige um formato específico cuja organização impacta diretamente a forma como essas instruções serão processadas. Com base na análise da posição dos operandos e nas semelhanças entre as instruções apresentadas na [Tabela 2](#), foram definidos os 6 formatos de instrução, os quais são detalhados a seguir.

Formato OPR

OPCODE	RD	RS	RT
[31:26]	[25:20]	[19:14]	[13:8]

Formato OPI

OPCODE	RD	RS	IM14/—
[31:26]	[25:20]	[19:14]	[13:0]

Formato B

OPCODE	RS	RT	IM14
[31:26]	[25:20]	[19:14]	[13:0]

Formato M

OPCODE	RD/RS	IM20/ENDEREÇO/PORTA/SWITCHES/—
[31:26]	[25:20]	[19:0]

Formato J

OPCODE	IM26
[31:26]	[25:0]

Formato C

OPCODE	—
[31:26]	[25:0]

Na [Tabela 3](#) abaixo, são apresentados os tipos de instruções e seus respectivos formatos. O Formato OPR corresponde às instruções do Tipo R, que realizam operações

lógicas (AND, OR e NOT) e também aritméticas (add, sub, mult e div). O Formato OPI é utilizado por algumas instruções do Tipo I, incluindo aquelas de acesso à memória com diferentes modos de endereçamento (como lwr, swr, lwd e swd) e operações de deslocamento, devido à similaridade na distribuição dos operandos. O Formato M é destinado às instruções de acesso à memória, operações de pilha, e também para entrada e saída (IN e OUT). Os Formatos B e J são aplicados às instruções do Tipo J, responsáveis por saltos condicionais e incondicionais. Finalmente, o Formato C é utilizado pelas instruções de controle, como HLT (para parar o processador) e NOP.

Tabela 3 – Formatos e Instruções Correspondentes

OPR	OPI	M	B	C	J
add	addi	li	beq	NOP	j
sub	subi	lw	bgt	HLT	jal
mult	multi	sw	blt		
div	ANDI	move	bge		
AND	ORI	jr	ble		
OR	sr	PUSH			
	sl	POP			
	lwr	IN			
	swr	OUT			
	lwd				
	swd				

4.3 Modos de Endereçamento

Os modos de endereçamento utilizados neste projeto foram selecionados com base naqueles apresentados na Fundamentação Teórica. Para exemplificar diferentes formas de acesso à memória, foram escolhidas duas instruções: **lw** (Load Word) e **sw** (Store Word). Cada uma delas será implementada com variações específicas de endereçamento, conforme detalhado a seguir.

4.3.1 Endereçamento Direto

As instruções que utilizam o modo de endereçamento direto são: **lw** e **sw**. Ambas foram definidas de forma que, no caso da instrução **lw**, seja fornecido um registrador de destino (RD) e um imediato de 20 bits, o qual representa diretamente o endereço de acesso à memória. Para a instrução **sw**, é fornecido um registrador de origem (RS), contendo o valor a ser armazenado, juntamente com um imediato de 20 bits que especifica o endereço de destino na memória. Ambas as instruções seguem o Formato M, conforme descrito na seção anterior.

4.3.2 Endereçamento Indireto por Registrador

As instruções que utilizam o modo de endereçamento indireto por registrador são: **lwr** e **swr**. Essas instruções são variações das instruções **lw** e **sw**, adaptadas para operar com dois registradores em vez de um imediato. Na instrução **lwr**, são fornecidos um registrador de destino (RD) e um registrador de origem (RS), sendo este último responsável por armazenar o endereço efetivo na memória de onde o dado será carregado para o RD. Já na instrução **swr**, também são utilizados dois registradores: o primeiro (RS) contém o valor a ser armazenado na memória, enquanto o segundo (RT) armazena o endereço efetivo de destino. Para ambas as instruções, adotou-se o Formato OPI, que admite dois campos fixos de registrador e um campo adicional opcional, que pode ou não conter um valor imediato.

4.3.3 Endereçamento Base-Deslocamento por Registrador

As instruções que empregam o modo de endereçamento por base com deslocamento são: **lwd** e **swd**. Essas instruções são extensões das instruções **lwr** e **swr**, incorporando, além do registrador base, um valor de deslocamento que será somado ao conteúdo do registrador para formar o endereço efetivo. Na instrução **lwd**, são fornecidos três operandos: um registrador de destino (RD), um registrador de base (RS) contendo o endereço base, e um valor imediato que representa o deslocamento a ser somado ao endereço base. O endereço resultante é utilizado para acessar a memória e carregar o valor correspondente para o registrador RD. De maneira análoga, na instrução **swd**, são utilizados um registrador de origem (RS), que contém o valor a ser armazenado, um registrador base (RT) que guarda o endereço base, e um imediato que será somado ao conteúdo de RT para formar o endereço efetivo de escrita na memória.

4.4 Caminho de Dados

Para a construção do caminho de dados foi levado em conta quais instruções serão executadas, ou seja, quais operações serão feitas pelo processador, e também, foi usado como referência a construção do caminho de dados base do MIPS, com as devidas adaptações, extraídas de (GIBBONS, n.d.), (GERSNOVIEZ et al., 2020) e (SANTOS, 2014).

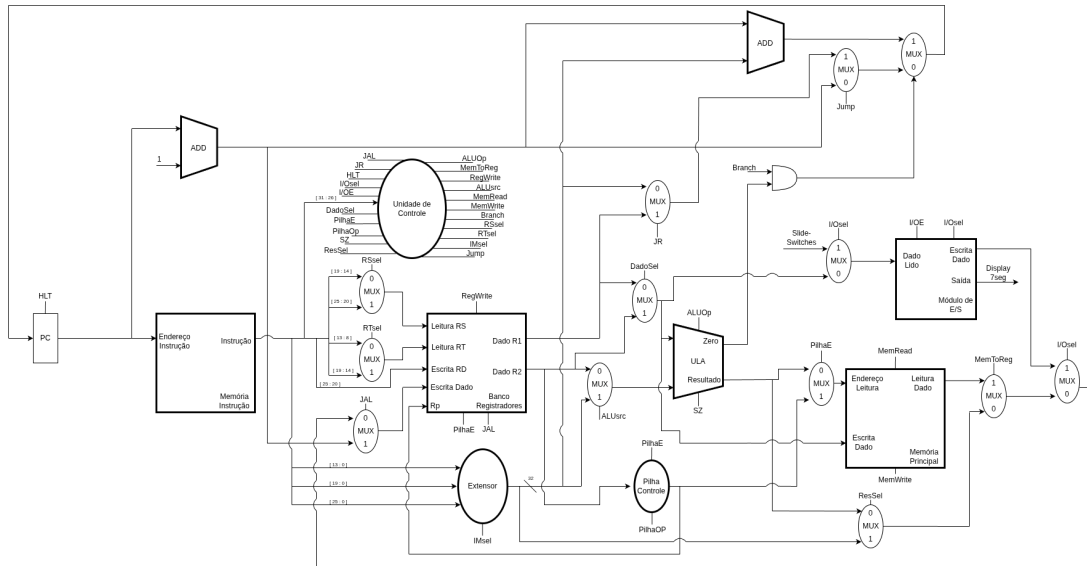


Figura 6 – Diagrama Datapath Geral do Processador

É possível observar que pela [Figura 6](#) como foi organizado o *datapath*, os componentes que compõe ele são:

- PC: Program Counter.
- Unidade de Controle: Controla sinais para habilitar e selecionar dados.
- ULA: Realiza operações aritméticas e lógicas no processador.
- Extensor de Bits: Estende entrada de imediatos de tamanhos diferentes para 32 bits.
- Banco de Registradores: Armazena todos os registradores de uso geral e específico.
- Memória de Instrução: Armazena as instruções que serão executadas pelo processador.
- Memória de Dados: Armazena os dados que serão buscados ou editados, fazendo leitura ou escrita.
- Pilha Controle: Controla o registrador de topo da pilha e realiza deslocamentos nele.
- Módulo de Entrada de Saída: Controla as instruções de IN e OUT, faz comunicação com os periféricos (Slide-Switches e Displays de 7 segmentos).
- Somadores: Fazem a operação de soma.
- Multiplexadores: Selecionam entradas.

Com esses elementos é possível realizar todas as instruções propostas. E para isso, são usados vários sinais de controle, com cada um para realizar alguma seleção de dados

ou habilitar algum componente ou ação. Abaixo é definido como cada sinal de controle opera.

Tabela 4 – Sinais de Controle e suas Funções

Sinal	Função
ALUOp	Determina a operação que a ULA deve executar (adição, subtração, AND, OR, etc.).
ALUsrc	Seleciona a origem do segundo operando da ULA: registrador ou imediato.
MemRead	Habilita a leitura de dados da memória.
MemWrite	Habilita a escrita de dados na memória.
MemToReg	Seleciona se o dado a ser escrito no registrador vem da memória ou da ULA.
RegWrite	Habilita a escrita em um registrador.
RSsel	Seleciona a origem do registrador RS.
RTsel	Seleciona a origem do registrador RT.
IMsel	Seleciona o tipo de imediato a ser estendido (13, 20 ou 26 bits).
Dadosel	Controla a origem dos registradores para fazer operações com diferentes endereçamentos (lwr/swr, ou lwd/swd).
ResSel	Seleciona qual resultado utilizar entre diferentes fontes (Resultado ULA ou Imediato).
Jump	Indica instrução de salto incondicional (jump).
JAL	Indica salto com link (<i>jump and link</i>), armazenando o endereço de retorno no registrador \$ra.
JR	Indica salto para o endereço contido em um registrador (<i>jump register</i>).
Branch	Indica salto condicional, baseado no resultado da ULA.
HLT	Sinaliza parada da CPU (<i>halt</i>).
PilhaE	Habilita o uso da pilha (push ou pop).
PilhaOp	Determina a operação da pilha: push (inserção) ou pop (remoção).
I/Osel	Seleciona se a operação de E/S (instrução IN ou OUT).
I/OE	Habilita a operação de entrada ou saída com dispositivos I/O.
SZ	Define soma da entrada com valor zero (cálculo de endereços).

4.4.1 Instrução *add* - Tipo R - Formato OPR

A execução da instrução *add* no *datapath* ocorre conforme os seguintes passos:

1. O PC busca a instrução a ser executada e, em seguida, é incrementado em 1.
2. A instrução é decodificada, identificando-se os campos RD (registrador de destino), RS (primeiro operando) e RT (segundo operando). Simultaneamente, a instrução é

encaminhada à Unidade de Controle, que ativa os seguintes sinais:

- **ALUOp**: seleciona a operação de soma na ULA;
- **RegWrite**: habilita a escrita no banco de registradores.

3. Os valores armazenados em RS e RT são somados pela ULA, e o resultado é escrito no registrador RD.

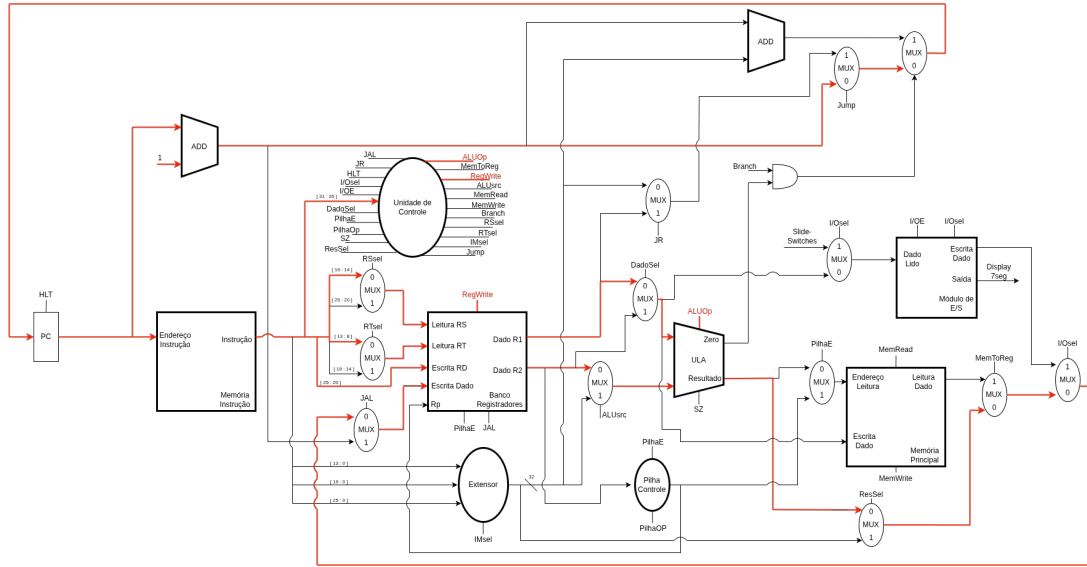


Figura 7 – Diagrama do Datapath da instrução *add*

Na Figura 7, é possível visualizar o caminho percorrido pelos dados (em vermelho), conforme descrito nos passos acima.

4.4.2 Instrução *addi* - Tipo I - Formato OPI

A execução da instrução *addi* segue os seguintes passos:

1. O PC busca a instrução e é incrementado em 1.
2. A instrução é decodificada em RD (registrador de destino), RS (registrador com o operando base) e um campo imediato de 14 bits, que é estendido para 32 bits para possibilitar a operação. A Unidade de Controle ativa os seguintes sinais:

- **ALUOp**: seleciona a operação de soma;
- **RegWrite**: habilita a escrita no banco de registradores;
- **ALUsrc**: seleciona o imediato como segundo operando da ULA;
- **IMsel**: define o tamanho do imediato a ser estendido.

3. A ULA realiza a soma entre o valor de RS e o imediato estendido. O resultado é então escrito no registrador RD.

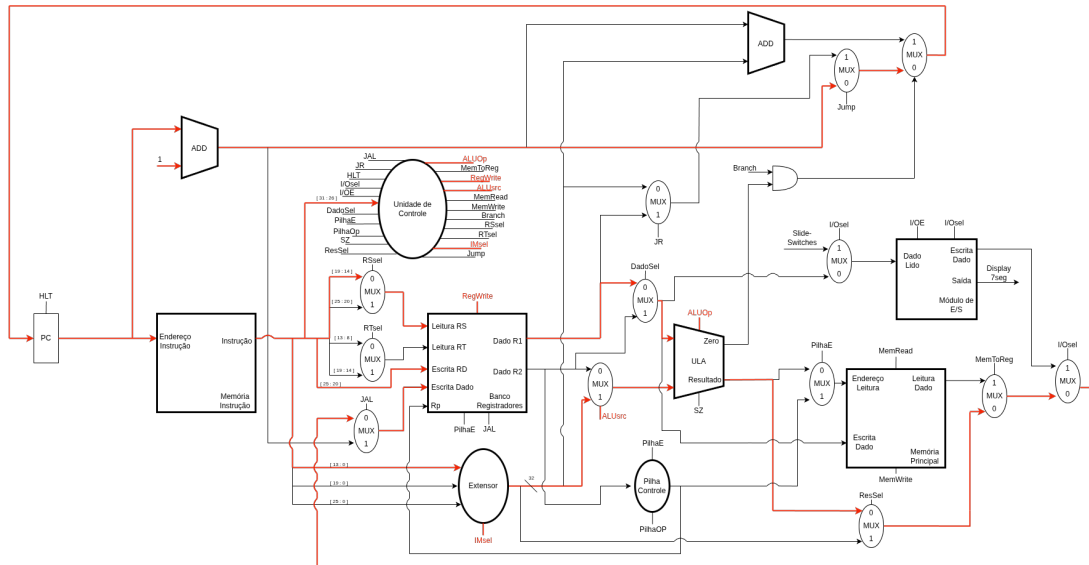


Figura 8 – Diagrama do Datapath da instrução *addi*

Na Figura 8, observa-se a seleção do imediato como operando para a ULA, bem como a decodificação dos campos da instrução.

4.4.3 Instrução *lwr* - Tipo R - Formato OPI

A instrução *lwr* é executada da seguinte maneira:

1. O PC busca a instrução e é incrementado em 1.
2. A instrução é decodificada nos campos RD (registrador de destino) e RS (registrador contendo o endereço base). Ao mesmo tempo, a Unidade de Controle ativa os seguintes sinais:
 - **ALUOp**: seleciona a operação de soma;
 - **RegWrite**: habilita a escrita no banco de registradores;
 - **MemToReg**: direciona o valor lido da memória para ser escrito no registrador;
 - **MemRead**: habilita a leitura da memória;
 - **SZ**: instrui a ULA a somar com o valor zero, mantendo o endereço base.
3. O endereço efetivo é calculado e acessado na memória. O dado lido é então encaminhado para o registrador RD.

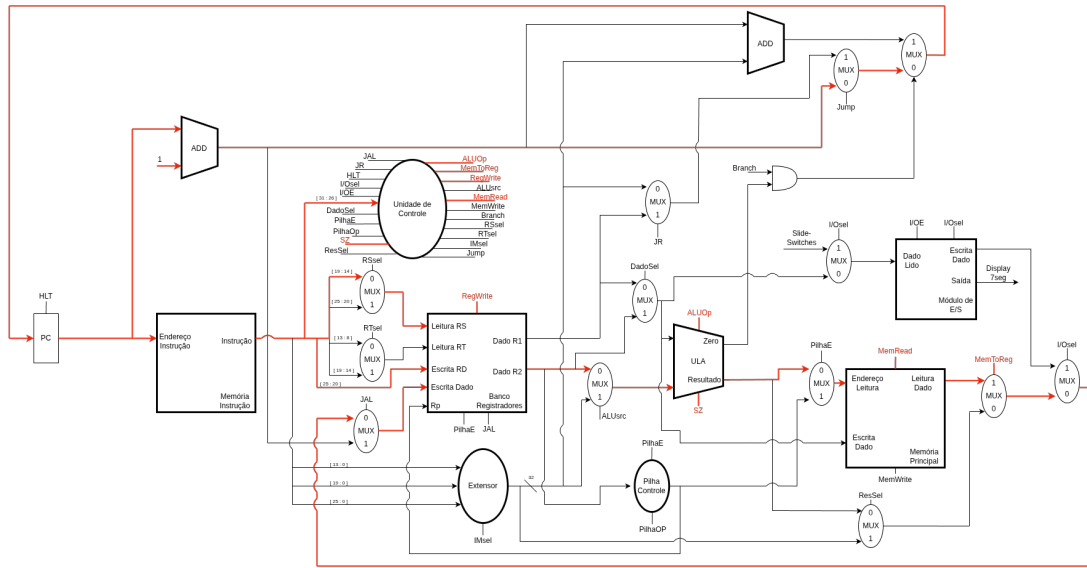


Figura 9 – Diagrama do Datapath da instrução *lwr*

Conforme a Figura 9, é possível observar o cálculo do endereço efetivo a partir do conteúdo de RS, bem como a leitura da memória e escrita no registrador RD.

4.4.4 Instrução *beq* - Tipo J - Formato B

A instrução *beq* é executada conforme os passos a seguir:

1. O PC busca a instrução e é incrementado em 1.
2. A instrução é decodificada nos campos RS e RT (registradores a serem comparados) e um campo imediato de 14 bits, utilizado para calcular o endereço de destino do salto. A Unidade de Controle ativa os seguintes sinais:
 - **ALUOp**: seleciona a operação de subtração para comparação;
 - **Branch**: habilita o salto condicional;
 - **RSsel** e **RTsel**: selecionam os registradores que serão comparados;
 - **IMsel**: define o tamanho do imediato a ser estendido.
3. Os valores de RS e RT são comparados pela ULA. Caso $RS = RT$, a saída **Zero** da ULA é ativada (valor 1), o que habilita o salto e atualiza o PC com o novo endereço.

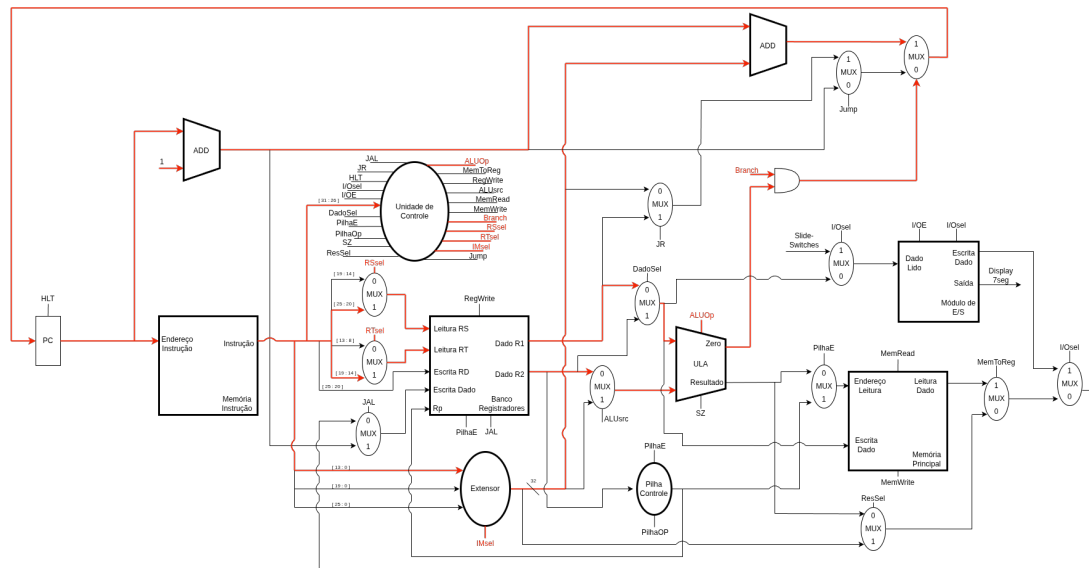


Figura 10 – Diagrama do Datapath da instrução *beq*

Na [Figura 10](#), observa-se a comparação entre os registradores e o desvio condicional do fluxo de execução, em caso de igualdade.

4.5 Implementação do Projeto

Nesta seção será discutido sobre as implementações dos códigos em Verilog HDL dos seguintes módulos do processador:

- ULA (Unidade Lógica Aritmética)
- Banco de Registradores
- Memória de Dados
- Memória de Instrução
- PC (Program Counter)
- Multiplexadores
- Extensor de Bits
- Unidade de Controle
- Unidade de Controle da Pilha
- Divisor de Frequência
- DBounce

- Módulo de Entrada e Saída
- Conversor para BCD
- Conversor BCD para 7 segmentos
- Integração

4.5.1 Unidade Lógica e Aritmética (ULA)

O projeto da ULA iniciou com uma análise detalhada do conjunto de instruções do processador, apresentado na [Tabela 2](#). O objetivo foi garantir que a ULA pudesse suportar todas as operações necessárias para a correta execução do programa. A partir dessa análise, foi definido o mapeamento entre cada função da ULA e um código binário específico para o sinal de controle **ALUOp**, conforme detalhado na [Tabela 5](#).

ALUOp	Operação
0000	add (adição)
0001	sub (subtração)
0010	mult (multiplicação)
0011	div (divisão)
0100	AND (E lógico)
0101	OR (OU lógico)
0110	NOT (NÃO lógico)
0111	beq (desvio se igual)
1000	bge (desvio se maior ou igual)
1001	ble (desvio se menor ou igual)
1010	blt (desvio se menor que)
1011	bgt (desvio se maior que)
1100	sl (deslocamento lógico para esquerda)
1101	sr (deslocamento lógico para direita)

Tabela 5 – Códigos de Operação da ULA (ALUOp)

```

1 module ULA(in1, in2, ALUOp, sz, branch, result);
2
3     input [31:0] in1; // Dado de Entrada 1
4     input [31:0] in2; // Dado de Entrada 2
5     input [3:0] ALUOp; // Sinal da Operacao da ULA
6     input sz; // Sinal para controlar casos de Endereco
7
8     output branch;
9     output [31:0] result;
10
11     reg [31:0] r;
12     reg b;
13

```

```

14 // ALUop codes
15 // add: 0000
16 // sub: 0001
17 // mult: 0010
18 // div: 0011
19 // and: 0100
20 // or: 0101
21 // not: 0110
22 // beq: 0111
23 // bge: 1000
24 // ble: 1001
25 // blt: 1010
26 // bgt: 1011
27 // sl: 1100
28 // sr: 1101
29
30 always @ (*) begin
31     // valores padrao
32     r = 32'b0;
33     b = 1'b0;
34
35     if (sz == 1'b1) begin
36         r = in2;
37     end
38
39     else begin
40         case (ALUop)
41             4'b0000: r = in1 + in2;
42             4'b0001: r = in1 - in2;
43             4'b0010: r = in1 * in2;
44             4'b0011: r = in1 / in2;
45             4'b0100: r = in1 & in2;
46             4'b0101: r = in1 | in2;
47             4'b0110: r = ~in1;
48             4'b0111: b = (in1 == in2);
49             4'b1000: b = (in1 >= in2);
50             4'b1001: b = (in1 <= in2);
51             4'b1010: b = (in1 < in2);
52             4'b1011: b = (in1 > in2);
53             4'b1100: r = in1 << 1; // shift left
54             4'b1101: r = in1 >> 1; // shift right
55             default:
56                 begin
57                     r = 32'b0;
58                     b = 1'b0;
59                 end
60         endcase
61     end
62 end
63
64 assign result = r;
65 assign branch = b;
66
67 endmodule

```

Código 4.1 – Código Verilog do Módulo ULA

O código apresentado no [Código 4.1](#) inicia com a declaração de suas portas de entrada – *in1* e *in2* para os operandos de 32 bits, *ALUOp* como o seletor de operação de

4 bits, e *sz* como um sinal de controle adicional de 1 bit. As portas de saída são *branch*, indicando o resultado de uma operação de desvio, e *result* para a saída principal de 32 bits da ULA. Internamente, são utilizados os sinais do tipo *reg*, *r* e *b*, para armazenar temporariamente os valores calculados para *result* e *branch*, respectivamente, dentro do bloco combinacional *always @ (*)*.

A funcionalidade central da ULA é determinada pelo sinal de controle *ALUOp*. Sua operação é definida em um bloco case que, para cada valor de *ALUOp*, executa uma operação aritmética (como adição, subtração, multiplicação, divisão), lógica bit a bit (AND, OR, NOT) ou de deslocamento (shift left - sl, shift right - sr). Para operações de comparação (como beq, bge, ble, blt, bgt), o resultado da comparação é atribuído ao sinal interno *b*, que posteriormente definirá a saída *branch*. Nas demais operações que produzem um resultado numérico, este é armazenado em *r*.

O sinal de controle *sz* possui um comportamento prioritário: quando *sz* está em nível lógico '1', a ULA efetivamente "ignora" a operação selecionada por *ALUOp* para o cálculo do resultado principal. Nesse caso, a entrada *in2* é diretamente passada para o registrador interno *r* (e, conseqüentemente, para a saída *result*). Esta característica pode ser empregada em cenários onde é necessário que um valor (potencialmente um endereço ou um dado imediato que não precise de processamento pela ULA) atravessasse o caminho de dados da ULA, o que é comum em diferentes modos de endereçamento ou para otimizar certas instruções. Se *sz* estiver em nível lógico '0', a ULA opera conforme a lógica do case baseada em *ALUOp*.

No início de cada avaliação do bloco *always @ (*)*, os sinais internos *r* e *b* são redefinidos para valores padrão (0 para *r* e '0' para *b*). Isso garante que, mesmo que uma operação específica não modifique um desses sinais (por exemplo, uma adição não modifica *b*, e uma comparação não modifica *r* dentro das cláusulas do case), eles terão um valor conhecido. Finalmente, as saídas *result* e *branch* do módulo são continuamente atribuídas (*assign*) com os valores finais de *r* e *b*.

4.5.2 Banco de Registradores

A implementação do Banco de Registradores foi projetada com base em três decisões da arquitetura projetada. Primeiramente, foram estabelecidos endereços fixos para registradores de propósito específico, como o ponteiro de pilha (*rp*) e o de endereço de retorno (*ra*), otimizando o acesso a eles, como pode ser visto na [Tabela 6](#).

A lógica de escrita é controlada por três sinais distintos: *RegWrite*, *PilhaE* e *JAL*. O sinal *RegWrite* habilita a escrita em registradores de uso geral. Em contrapartida, *PilhaE* e *JAL* gerenciam operações dedicadas: *PilhaE* atualiza o registrador do topo da pilha, enquanto *JAL* salva o endereço de retorno para a instrução de desvio e link (*Jump*

and Link).

Por fim, a temporização do módulo distingue as operações: a escrita é um processo síncrono, ocorrendo na borda de subida do clock para garantir estabilidade. A leitura, por sua vez, é implementada de forma combinacional (assíncrona), permitindo que os dados fiquem disponíveis instantaneamente, independentemente dos ciclos de clock.

Endereço	Registrador
000000	\$zero
000001	\$ra
000011	\$rp

Tabela 6 – Endereços dos Registradores Específicos

```

1 module BancoRegistradores(clock, addr_rs, addr_rt, addr_rd, escrita_dado, rp, RegWrite,
  PilhaE, JAL, dado1, dado2);
2
3     input clock;
4
5     input [5:0]addr_rs; // Endere o do registrador de entrada RS
6     input [5:0]addr_rt; // Endere o do registrador de entrada RT
7     input [5:0]addr_rd; // Endere o do registrador de entrada RD
8     input [31:0]escrita_dado; // Valor do Dado que veio da ULA ou da Mem ria
9     input [31:0]rp; // Novo endere o do $rp calculado da UC da Pilha
10
11     // Sinais de Controle
12     input RegWrite;
13     input PilhaE;
14     input JAL;
15
16     output [31:0]dado1; // Sa da do registrador RS
17     output [31:0]dado2; // Sa da do registrador RT
18
19     // Posi o para Registradores especificos:
20     // $zero = 000000
21     // $ra = 000001
22     // $rp = 000010
23     parameter addr_zero = 6'b000000;
24     parameter addr_ra = 6'b000001;
25     parameter addr_rp = 6'b000011;
26
27     reg p_clk;
28
29     reg [31:0]banco[63:0]; // Banco de Registradores 64x32
30
31     initial begin
32         p_clk = 1'b1;
33     end
34
35     // inicializa o endereco salvo no $rp que sera usado como topo da pilha na
      mem ria
36     always @ (negedge clock) begin
37

```

```

38         banco[addr_zero] = 32'd0;
39
40         if (p_clk) begin
41             banco[addr_rp] = 32'd25;
42             p_clk = 1'b0;
43         end
44
45         if (RegWrite) begin
46             banco[addr_rd] = escrita_dado;
47         end
48
49         if (PilhaE) begin
50             banco[addr_rp] = rp;
51         end
52
53         if (JAL) begin
54             banco[addr_ra] = escrita_dado;
55         end
56
57     end
58
59     assign dado1 = banco[addr_rs];
60     assign dado2 = PilhaE ? banco[addr_rp] : banco[addr_rt];
61
62
63 endmodule

```

Código 4.2 – Código Verilog do Módulo Banco de Registradores

A implementação do código apresentado no [Código 4.2](#) utiliza uma estrutura de memória síncrona para escrita e combinacional para leitura. Uma característica notável é a declaração do banco como `reg [63:0] banco[31:0]`, que define 64 registradores de 32 bits. A lógica de leitura é direta para a porta `dado1`, que acessa o endereço em `addr_rs`, mas é condicional para `dado2`. A porta `dado2` lê do ponteiro de pilha (`rp`) quando o sinal de controle `PilhaE` está ativo; caso contrário, lê do endereço em `addr_rt`, criando um caminho de dados especializado para operações de pilha.

A lógica de controle de escrita, que opera na borda de subida do clock dentro do bloco `always @ (posedge clock)`, é segmentada por função através dos sinais `RegWrite`, `PilhaE` e `JAL`. O sinal `RegWrite` habilita escritas de propósito geral no registrador especificado por `addr_rd`. Em paralelo, `PilhaE` e `JAL` acionam escritas dedicadas e com prioridade implícita aos registradores de função específica `$rp` (endereço 3) e `$ra` (endereço 1), respectivamente. O módulo garante estados iniciais e constantes importantes: o registrador `$rp` é inicializado com o valor '224' para definir a base da pilha, enquanto o registrador `$zero` é continuamente forçado a '0' em cada ciclo de clock, garantindo a integridade da constante zero.

4.5.3 Memória de Dados (RAM)

O planejamento para uma memória de dados em um processador inicia com a necessidade de armazenar e recuperar valores durante a execução do programa, o que define sua natureza como uma RAM (Memória de Acesso Aleatório). As operações essenciais são leitura e escrita, controladas por sinais específicos (**MemRead** e **MemWrite**). A escrita deve ser síncrona ao clock do sistema para garantir a integridade dos dados e evitar condições de corrida, ocorrendo apenas quando **MemWrite** está ativo. A leitura, por sua vez, pode ser implementada de forma combinacional para simplificar o acesso e disponibilizar o dado requisitado rapidamente. Caso o sinal **MemRead** não esteja ativo, a saída deve apresentar um valor padrão, como zero, para evitar a propagação de estados indefinidos. A flexibilidade é alcançada através da parametrização da largura dos dados (**DATA_WIDTH**) e da largura do barramento de endereços (**ADDR_WIDTH**).

```

1 module MemoriaDados
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=6)
3   (
4     input [(DATA_WIDTH-1):0] data, // Dado a ser escrito na Memoria
5     input [(ADDR_WIDTH-1):0] addr, // Endereco de escrita na Memoria
6     input MemWrite, MemRead, clk,
7     output [(DATA_WIDTH-1):0] q // Saida de leitura da Memoria
8   );
9
10  // Declaracao da Memoria de Dados
11  reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];
12
13  always @ (posedge clk) begin
14    // Escrita Memoria
15    if (MemWrite)
16      ram[addr] <= data;
17  end
18
19  assign q = MemRead ? ram[addr] : 32'b0;
20
21 endmodule

```

Código 4.3 – Código Verilog do Módulo Memória de Dados

O módulo no [Código 4.3](#) é parametrizado por **DATA_WIDTH** e **ADDR_WIDTH**, definindo uma memória interna **ram** como um matriz de $2^{ADDR_WIDTH} = 256$ linhas e contendo um valor de **DATA_WIDTH** padronizado para 32 bits. A operação de escrita é síncrona, ocorrendo na borda de subida do **clk** e condicionada ao sinal **MemWrite**, quando ativo, o valor da entrada **data** é armazenado na posição **ram[addr]**. A lógica de leitura é combinacional: a saída **q** recebe o valor de **ram[addr]** se **MemRead** estiver ativo. Caso **MemRead** esteja inativo, a saída **q** é explicitamente definida como **32'b0**, garantindo um comportamento previsível para a leitura desabilitada.

4.5.4 Memória de Instrução (ROM)

A memória de instruções é concebida como uma ROM (Memória Somente de Leitura), pois as instruções do programa são carregadas no início e, tipicamente, não são alteradas durante a execução normal do processador. A sua função principal é fornecer a instrução apontada pelo Contador de Programa (PC). A leitura das instruções deve ser um processo combinacional, garantindo que a instrução no endereço fornecido (`addr_pc`) esteja disponível o mais rápido possível para a unidade de controle. Um mecanismo de inicialização é crucial para carregar o código do programa na memória no início da simulação ou durante a configuração do hardware. Assim como a memória de dados, a parametrização da largura dos dados (instrução) e do endereço é importante para adaptabilidade.

```
1 module MemoriaInstrucoes
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=4)
3   (
4     input [(ADDR_WIDTH-1):0] addr_pc, // Endereco de busca da Instrucao
5     output reg [(DATA_WIDTH-1):0] q // Saida com a Instrucao a ser executada
6   );
7
8   // Declaracao Memoria de Instrucoes
9   reg [DATA_WIDTH-1:0] rom[2**ADDR_WIDTH-1:0];
10
11   initial begin
12     $readmemb("codigo_teste.txt", rom);
13   end
14
15   always @ (*) begin
16     q = rom[addr_pc];
17   end
18
19 endmodule
```

Código 4.4 – Código Verilog do Módulo Memória de Instruções

O módulo no [Código 4.4](#), também parametrizado por `DATA_WIDTH` e `ADDR_WIDTH`, declara uma memória interna `rom`. Um bloco `initial` é utilizado para carregar o conteúdo da memória a partir de um arquivo externo, `"codigo_teste.txt"`, o qual conterá as instruções a serem executadas pelo processador, usando a diretiva de sistema `$readmemb`. Este processo ocorre apenas uma vez, no início da simulação. A lógica de leitura é implementada através de um bloco `always @ (*)`, que é sensível a qualquer mudança na entrada `addr_pc`. Dentro deste bloco, a saída `q` (declarada como `output reg`) é continuamente atualizada com o conteúdo de `rom[addr_pc]`, caracterizando uma leitura combinacional.

4.5.5 Registrador Contador de Programa (PC)

O projeto de um Contador de Programa (PC) parte de sua função primária: armazenar o endereço da próxima instrução a ser executada. Para isso, é necessário um

registrador interno para guardar o endereço atual. A atualização desse valor deve ser síncrona com o clock do sistema para garantir a ordem correta da execução. A lógica de atualização precisa suportar duas operações fundamentais: o incremento sequencial (para buscar a próxima instrução, como $PC \leftarrow PC + 1$) e o carregamento de um novo endereço vindo de desvios (branches) ou saltos (jumps). Além disso, um sinal de controle para pausar ou "congelar" o PC é essencial para finalizar o processamento. Para as instruções de Entrada e Saída existem dois sinais *confirm* e *stall*, para a instrução *IN* o PC deve parar para poder esperar o usuário inserir a entrada, ou seja, o *stall* liga para esperar a confirmação do usuário com o *confirm*.

```

1 module PC (
2     input clock,
3     input reset,
4     input HLT,
5     input stall,
6     input confirm,
7     input [31:0] in_pc,
8     output [31:0] out_pc
9 );
10
11     reg [31:0] pc;
12
13     initial begin
14         pc = 32'b0;
15     end
16
17     always @(posedge clock or posedge reset) begin
18         if (reset) begin
19             pc <= 32'b0;
20         end
21         else if (HLT) begin
22             pc <= pc;
23         end
24         else if (stall) begin
25             if (confirm) begin
26                 pc <= in_pc;
27             end
28             else begin
29                 pc <= pc;
30             end
31         end
32         else begin
33             pc <= in_pc;
34         end
35     end
36
37     assign out_pc = pc;
38
39 endmodule

```

Código 4.5 – Código Verilog do Módulo PC

O módulo que pode ser visto no [Código 4.5](#) implementa um registrador de 32 bits, *pc*, que armazena o endereço da instrução. Toda a lógica de escrita é contida em um bloco

`always @ (posedge clock)`, garantindo que as atualizações ocorram apenas na borda de subida do clock. A lógica é controlada pelo sinal `HLT`: se `HLT` estiver ativo (em nível lógico 1), o PC recarrega seu próprio valor (`pc <= pc`), efetivamente mantendo seu estado atual. Caso contrário, o PC é atualizado com o valor da entrada `in_pc`. Esta entrada externa abstrai a lógica de seleção entre o próximo endereço sequencial (`PC+1`) e um endereço de desvio, simplificando o módulo. Por fim, a saída `out_pc` expõe continuamente o valor armazenado no registrador `pc`.

4.5.6 Multiplexadores

O projeto de um multiplexador (MUX) baseia-se em sua função essencial de atuar como uma chave seletora digital. O objetivo é selecionar uma, e apenas uma, entre várias entradas de dados e direcioná-la para uma única saída.

```
1 module mux (a, b, s, out);  
2  
3     input [31:0] a;  
4     input [31:0] b;  
5     input s;  
6  
7     output [31:0] out;  
8  
9     assign out = s ? b : a;  
10 endmodule
```

Código 4.6 – Código Verilog do Módulo MUX

O módulo no [Código 4.6](#) implementa um multiplexador de duas entradas. Ele possui as entradas `a` e `b` como fontes de dados e a entrada `s` como o sinal de seleção. A lógica é se o sinal `s` for avaliado como verdadeiro (1), a saída `out` recebe o valor de `b`; caso contrário (0), `out` recebe o valor de `a`. Por ser um bloco puramente combinacional, qualquer alteração nas entradas `a`, `b` ou `s` é refletida imediatamente na saída `out`.

4.5.7 Somador

O Módulo Somador é um dos blocos lógicos mais fundamentais em qualquer arquitetura de computador, sendo a base para operações aritméticas. Sua função primordial é realizar a adição de dois números binários de entrada, produzindo um resultado que representa a soma desses valores. Este componente é amplamente utilizado na Unidade Lógica e Aritmética (ULA) de um processador, bem como em cálculos de endereços de memória e outras operações que exigem adição.

```
1 module Adder (a, b, out);  
2  
3     input [31:0] a;  
4     input [31:0] b;  
5  
6     output reg [31:0] out;
```

```
7
8 always @ (*) begin
9
10     out = a + b;
11
12 end
13
14 endmodule
```

Código 4.7 – Código Verilog do Módulo Somador

Conforme ilustrado no [Código 4.7](#), o módulo Adder recebe duas entradas de 32 bits, **a** e **b**, que representam os operandos a serem somados. A saída, **out**, também de 32 bits, armazena o resultado da operação de adição. A lógica de desenvolvimento é direta e implementada dentro de um bloco `always @ (*)`, que indica que a saída **out** é atualizada sempre que qualquer uma das entradas (**a** ou **b**) muda. A operação de adição é realizada simplesmente utilizando o operador `+` do Verilog (`out = a + b;`), que executa uma adição binária de 32 bits. Por ser um módulo puramente combinacional, o resultado da soma é instantaneamente refletido na saída **out** após qualquer alteração nas entradas.

4.5.8 Extensor de Bits

O Módulo Extensor de Bits é um componente crucial em arquiteturas de processadores, especialmente quando se lida com instruções que utilizam operandos de diferentes tamanhos ou valores imediatos. Sua principal função é estender um valor de entrada, que possui um número menor de bits, para um formato de saída com um número maior de bits, geralmente preenchendo os bits mais significativos com zeros (zero-extension) ou com o bit de sinal (sign-extension), dependendo da aplicação. No contexto deste módulo, a extensão é feita para 32 bits, permitindo que valores imediatos de diferentes tamanhos sejam utilizados em operações de 32 bits.

```
1 module Extensor (in1, in2, in3, IMsel, out);
2
3 input [13:0] in1;
4 input [19:0] in2;
5 input [25:0] in3;
6 input [1:0] IMsel;
7
8 output reg [31:0] out;
9
10
11 always @ (*) begin
12
13     if (IMsel == 2'b00) begin
14         out = in1 + 32'd0;
15     end
16     if (IMsel == 2'b01) begin
17         out = in2 + 32'd0;
18     end
19     if (IMsel == 2'b10) begin
20         out = in3 + 32'd0;
```



```

21     end
22 end
23
24 endmodule

```

Código 4.8 – Código Verilog do Módulo Extensor

Conforme ilustrado no [Código 4.8](#), o módulo Extensor possui três entradas de dados com diferentes larguras de bits: `in1` (14 bits), `in2` (20 bits) e `in3` (26 bits). A entrada `IMsel` (Immediate Select), de 2 bits, atua como um seletor para determinar qual das entradas de dados será estendida. A saída, `out`, é um registrador de 32 bits que armazena o valor estendido.

A lógica do módulo é implementada dentro de um bloco `always @ (*)`, tornando-o um circuito puramente combinacional. A seleção e extensão ocorrem da seguinte forma:

- Se `IMsel` for `2'b00`, a entrada `in1` é estendida para 32 bits e atribuída a `out`. A operação `+ 32'd0` em Verilog é uma forma concisa de realizar a extensão de zero, onde os bits mais significativos de `in1` são preenchidos com zeros até completar 32 bits.
- Se `IMsel` for `2'b01`, a entrada `in2` é estendida para 32 bits e atribuída a `out`.
- Se `IMsel` for `2'b10`, a entrada `in3` é estendida para 32 bits e atribuída a `out`.

É importante notar que, para o caso em que `IMsel` é `2'b11`, a saída `out` manterá seu valor anterior, pois não há uma atribuição explícita para essa condição. Este módulo permite que o processador utilize valores imediatos de diferentes tamanhos de forma flexível, adaptando-os ao formato de dados padrão de 32 bits.

4.5.9 Unidade de Controle

A Unidade de Controle (UC) é o "cérebro" de um processador, responsável por coordenar todas as operações e garantir que as instruções sejam executadas corretamente. Sua função primordial é decodificar as instruções (opcodes) e gerar os sinais de controle apropriados para todos os outros componentes do datapath, como a Unidade Lógica e Aritmética (ULA), o banco de registradores, a memória e os módulos de entrada/saída. A UC é um circuito sequencial que define o comportamento do processador em cada ciclo de clock, controlando o fluxo de dados e as operações realizadas.

```

1 module UnidadeControle (opcode, JAL, JR, HLT, DadoSel, PilhaE, PilhaOP, SZ,
2   ResSel, ALUOp, MemToReg, RegWrite, ALUSrc, MemRead,
3   MemWrite, Branch, RSsel, RTsel, IMsel, Jump, IOE, IOSel, stall);
4
5   input [5:0] opcode;
6
7   output reg SZ;

```

```

8  output reg JAL;
9  output reg JR;
10 output reg HLT;
11 output reg Jump;
12 output reg Branch;
13 output reg RegWrite;
14 output reg RSsel;
15 output reg RTsel;
16 output reg DadoSel;
17 output reg ResSel;
18 output reg PilhaE;
19 output reg PilhaOP;
20 output reg ALUsrc;
21 output reg MemToReg;
22 output reg MemRead;
23 output reg MemWrite;
24 output reg IOE;
25 output reg IOsel;
26 output reg stall;
27 output reg [1:0] IMsel;
28 output reg [3:0] ALUOp;
29
30 always @ (*) begin
31     JAL = 0;
32     JR = 0;
33     HLT = 0;
34     DadoSel = 0;
35     PilhaE = 0;
36     PilhaOP = 0;
37     SZ = 0;
38     ResSel = 0;
39     MemToReg = 0;
40     RegWrite = 0;
41     ALUsrc = 0;
42     MemRead = 0;
43     MemWrite = 0;
44     Branch = 0;
45     RSsel = 0;
46     RTsel = 0;
47     Jump = 0;
48     IMsel = 2'b00;
49     ALUOp = 4'b0000;
50     IOE = 0;
51     IOsel = 0;
52     stall = 0;
53
54     case (opcode)
55         // add
56         6'b000000: begin
57             ALUOp = 4'b0000;
58             RegWrite = 1;
59         end
60         // sub
61         6'b000001: begin
62             ALUOp = 4'b0001;
63             RegWrite = 1;
64         end
65         // mult
66         6'b000010: begin
67             ALUOp = 4'b0010;

```

```
68         RegWrite = 1;
69     end
70     // div
71     6'b000011:begin
72         ALUOp = 4'b0011;
73         RegWrite = 1;
74     end
75     // AND
76     6'b000100:begin
77         ALUOp = 4'b0100;
78         RegWrite = 1;
79     end
80     // OR
81     6'b000101:begin
82         ALUOp = 4'b0101;
83         RegWrite = 1;
84     end
85     // NOT
86     6'b000110:begin
87         ALUOp = 4'b0110;
88         RegWrite = 1;
89     end
90     // addi
91     6'b000111:begin
92         ALUOp = 4'b0000;
93         RegWrite = 1;
94         IMsel = 2'b00;
95         ALUsrc = 1;
96     end
97     // subi
98     6'b001000:begin
99         ALUOp = 4'b0001;
100        RegWrite = 1;
101        IMsel = 2'b00;
102        ALUsrc = 1;
103    end
104    // multi
105    6'b001001:begin
106        ALUOp = 4'b0010;
107        RegWrite = 1;
108        IMsel = 2'b00;
109        ALUsrc = 1;
110    end
111    // ANDI
112    6'b001010:begin
113        ALUOp = 4'b0100;
114        RegWrite = 1;
115        IMsel = 2'b00;
116        ALUsrc = 1;
117    end
118    // ORI
119    6'b001011:begin
120        ALUOp = 4'b0101;
121        RegWrite = 1;
122        IMsel = 2'b00;
123        ALUsrc = 1;
124    end
125    // sr
126    6'b001100:begin
127        ALUOp = 4'b1101;
```

```

128         RegWrite = 1;
129     end
130     // sl
131     6'b001101:begin
132         ALUOp = 4'b1100;
133         RegWrite = 1;
134     end
135     // bge
136     6'b001110:begin
137         ALUOp = 4'b1000;
138         Branch = 1;
139         IMsel = 2'b01;
140         RSsel = 1;
141         RTsel = 1;
142     end
143     // beq
144     6'b001111:begin
145         ALUOp = 4'b0111;
146         Branch = 1;
147         IMsel = 2'b01;
148         RSsel = 1;
149         RTsel = 1;
150     end
151     // bgt
152     6'b010000:begin
153         ALUOp = 4'b1011;
154         Branch = 1;
155         IMsel = 2'b01;
156         RSsel = 1;
157         RTsel = 1;
158     end
159     // blt
160     6'b010001:begin
161         ALUOp = 4'b1010;
162         Branch = 1;
163         IMsel = 2'b01;
164         RSsel = 1;
165         RTsel = 1;
166     end
167     // ble
168     6'b010010:begin
169         ALUOp = 4'b1001;
170         Branch = 1;
171         IMsel = 2'b01;
172         RSsel = 1;
173         RTsel = 1;
174     end
175     // move
176     6'b010011:begin
177         SZ = 1;
178         RegWrite = 1;
179         RTsel = 1;
180     end
181     // li
182     6'b010100:begin
183         SZ = 1;
184         RegWrite = 1;
185         IMsel = 2'b01;
186         ALUsrc = 1;
187     end

```

```

188         // lw
189         6'b010101:begin
190             SZ = 1;
191             RegWrite = 1;
192             IMsel = 2'b01;
193             ALUsrc = 1;
194             MemRead = 1;
195             MemToReg = 1;
196         end
197         // sw
198         6'b010110:begin
199             SZ = 1;
200             RSsel = 1;
201             IMsel = 2'b01;
202             ALUsrc = 1;
203             MemWrite = 1;
204         end
205         // lwr
206         6'b010111:begin
207             ALUOp = 4'b0000;
208             RegWrite = 1;
209             MemRead = 1;
210             MemToReg = 1;
211         end
212         // swr
213         6'b011000:begin
214             ALUOp = 4'b0000;
215             RSsel = 1;
216             RTsel = 1;
217             MemWrite = 1;
218         end
219         // lwd
220         6'b011001:begin
221             ALUOp = 4'b0000;
222             ALUsrc = 1;
223             IMsel = 2'b00;
224             MemRead = 1;
225             RegWrite = 1;
226             MemToReg = 1;
227         end
228         // swd
229         6'b011010:begin
230             ALUOp = 4'b0000;
231             ALUsrc = 1;
232             IMsel = 2'b00;
233             RSsel = 1;
234             RTsel = 1;
235             MemWrite = 1;
236         end
237         // j
238         6'b011011:begin
239             Jump = 1;
240             IMsel = 2'b10;
241         end
242         // jrr
243         6'b011100:begin
244             RSsel = 1;
245             Jump = 1;
246             JR = 1;
247         end

```

```

248 // jal
249 6'b011101:begin
250     JAL = 1;
251     IMsel = 2'b10;
252     Jump = 1;
253 end
254 // PUSH
255 6'b011110:begin
256     RSsel = 1;
257     PilhaE = 1;
258     PilhaOP = 1;
259     MemWrite = 1;
260 end
261 // POP
262 6'b011111:begin
263     PilhaE = 1;
264     PilhaOP = 1;
265     MemRead = 1;
266     MemToReg = 1;
267 end
268
269 // IN
270 6'b100000:begin
271     IOE = 1;
272     IOsel = 1;
273     stall = 1;
274     RegWrite = 1;
275 end
276
277 // OUT
278 6'b100001:begin
279     IOE = 1;
280     IOsel = 0;
281     RSsel = 1;
282 end
283
284 // HLT
285 6'b100011:begin
286     HLT = 1;
287 end
288
289 default: begin
290     JAL = 0;
291     JR = 0;
292     HLT = 0;
293     DadoSel = 0;
294     PilhaE = 0;
295     PilhaOP = 0;
296     SZ = 0;
297     ResSel = 0;
298     MemToReg = 0;
299     RegWrite = 0;
300     ALUsrc = 0;
301     MemRead = 0;
302     MemWrite = 0;
303     Branch = 0;
304     RSsel = 0;
305     RTsel = 0;
306     Jump = 0;
307     IMsel = 2'b00;

```

```
308             ALUOp = 4'b0000;  
309             IOE    = 0;  
310             IOsel  = 0;  
311             stall = 0;  
312         end  
313     endcase  
314 end  
315  
316 endmodule
```

Código 4.9 – Código Verilog do Módulo Unidade de Controle

Conforme ilustrado no [Código 4.9](#), o módulo UnidadeControle recebe uma entrada de 6 bits, `opcode`, que representa o código de operação da instrução a ser executada. As saídas são uma série de sinais de controle de 1 bit (como `RegWrite`, `MemRead`, `Branch`, `Jump`, etc.) e alguns sinais de múltiplos bits, como `IMsel` (2 bits) e `ALUOp` (4 bits).

A lógica do módulo é implementada dentro de um bloco `always @ (*)`, tornando-o um circuito puramente combinacional. No início de cada avaliação, todos os sinais de controle são inicializados para seus valores padrão (geralmente 0 ou um estado neutro). Em seguida, uma estrutura `case` é utilizada para decodificar o `opcode` e gerar os sinais de controle específicos para cada instrução.

Por exemplo:

- Para instruções aritméticas como `add` (6'b000000), `ALUOp` é definido para 4'b0000 (código para adição na ULA) e `RegWrite` é ativado para permitir a escrita no banco de registradores.
- Para instruções com imediato, como `addi` (6'b000111), além de definir `ALUOp` e `RegWrite`, `IMsel` é configurado para 2'b00 (selecionando o imediato de 14 bits) e `ALUsrc` é ativado para que a ULA use o valor imediato.
- Instruções de desvio condicional, como `beq` (6'b001111), definem `ALUOp` para a comparação de igualdade, ativam `Branch` e configuram `IMsel` para 2'b01 (selecionando o imediato de 20 bits para o deslocamento do branch).
- Instruções de carga (`lw`) e armazenamento (`sw`) de memória ativam `MemRead` ou `MemWrite`, respectivamente, e configuram `ALUsrc` e `IMsel` para calcular o endereço de memória.
- As instruções de pilha, `PUSH` e `POP`, ativam `PilhaE` e `PilhaOP`, além de `MemWrite` ou `MemRead`, respectivamente.

- As instruções de I/O, IN e OUT, ativam `I0E` e `I0sel` para controlar o módulo de entrada/saída. A instrução IN também ativa `stall` para pausar o pipeline enquanto a entrada é aguardada.
- A instrução HLT (6'b100011) ativa o sinal HLT, que pode ser usado para parar a execução do processador.

Se um opcode não reconhecido for recebido, todos os sinais de controle são resetados para seus valores padrão, garantindo um estado seguro. O desenvolvimento deste módulo é crítico para a funcionalidade do processador, pois ele orquestra as operações de todos os outros componentes com base nas instruções do programa.

4.5.10 Unidade de Controle da Pilha

A Unidade de Controle da Pilha (UC_Pilha) é um módulo especializado responsável por gerenciar as operações de empilhamento (PUSH) e desempilhamento (POP) em uma estrutura de dados de pilha. Em arquiteturas de computador, a pilha é uma área de memória crucial para o gerenciamento de chamadas de sub-rotinas, passagem de parâmetros e armazenamento temporário de dados. Este módulo garante que o ponteiro da pilha seja atualizado corretamente e que os endereços de memória para as operações de leitura e escrita na pilha sejam gerados de forma precisa.

```

1 module UC_Pilha (rp_in, PilhaE, PilhaOP, rp_out, rp_mem);
2
3 input [31:0] rp_in;
4 input PilhaE;
5 input PilhaOP;
6
7 output reg [31:0] rp_out; // Volta para banco
8 output reg [31:0] rp_mem; // Acessa a memoria
9
10 always @ (*) begin
11
12     if (PilhaE) begin
13         if (PilhaOP) begin
14             // Push
15             rp_out = rp_in + 32'd1;
16             rp_mem = rp_in + 32'd1;
17         end
18     else begin
19         // Pop
20         rp_out = rp_in - 32'd1;
21         rp_mem = rp_in;
22     end
23 end
24 end
25
26 endmodule

```

Código 4.10 – Código Verilog do Módulo Unidade de Controle da Pilha

Conforme ilustrado no [Código 4.10](#), o módulo UC_Pilha recebe como entradas: `rp_in` (32 bits), que é o valor atual do ponteiro da pilha; `PilhaE` (1 bit), que habilita as operações da pilha; e `PilhaOP` (1 bit), que seleciona a operação a ser realizada (PUSH ou POP). As saídas são: `rp_out` (32 bits), que é o novo valor do ponteiro da pilha a ser atualizado no banco de registradores; e `rp_mem` (32 bits), que é o endereço de memória a ser acessado para a operação de pilha.

A lógica do módulo é implementada dentro de um bloco `always @ (*)`, indicando que é um circuito puramente combinacional. A operação é condicionada pelo sinal `PilhaE`. Se `PilhaE` estiver ativo, o módulo verifica o sinal `PilhaOP`:

- Se `PilhaOP` for verdadeiro (1), a operação é de PUSH. Nesse caso, o ponteiro da pilha é incrementado em 1 (`rp_in + 32'd1`), e esse novo valor é atribuído tanto a `rp_out` (para atualização do ponteiro) quanto a `rp_mem` (como o endereço onde o dado será armazenado).
- Se `PilhaOP` for falso (0), a operação é de POP. Para POP, o ponteiro da pilha é decrementado em 1 (`rp_in - 32'd1`), e esse novo valor é atribuído a `rp_out`. O endereço de memória para leitura (`rp_mem`) permanece o valor atual de `rp_in`, pois o dado a ser "desempilhado" está no endereço apontado antes do decremento.

Este módulo é fundamental para a correta manipulação da pilha, garantindo que o ponteiro da pilha seja sempre consistente com as operações de PUSH e POP, e que os acessos à memória para essas operações ocorram nos endereços corretos.

4.5.11 Divisor de Frequência

O Módulo Divisor de Frequência é um componente essencial em sistemas digitais, utilizado para gerar um sinal de clock com uma frequência menor a partir de um clock de entrada de frequência mais alta. Essa funcionalidade é crucial em diversas aplicações, como a sincronização de diferentes partes de um circuito que operam em velocidades distintas, ou para gerar sinais de temporização específicos para periféricos. O objetivo principal é criar um novo sinal de clock que tenha um período maior (e, conseqüentemente, uma frequência menor) do que o clock original, mantendo a estabilidade e a precisão.

```

1 module Divisor_Freq(
2   input clock_in,
3   input reset,
4   output reg clock_out
5 );
6
7 reg [25:0] count;
8
9 always @ (posedge clock_in) begin
10     if (reset) begin
11         count <= 26'd0;

```

```
12         clock_out <= 1'b0;
13     end
14     else begin
15         if (count == 26'd50) begin
16             count <= 26'd0;
17             clock_out = ~clock_out;
18         end
19         else begin
20             count <= count + 26'd1;
21         end
22     end
23 end
24
25 endmodule
```

Código 4.11 – Código Verilog do Módulo Divisor de Frequência

Conforme ilustrado no [Código 4.11](#), o módulo `Divisor_Freq` possui uma entrada de clock principal, `clock_in`, um sinal de reset assíncrono, `reset`, e uma saída de clock dividida, `clock_out`. Internamente, o módulo utiliza um registrador de 26 bits, `count`, que atua como um contador.

A lógica do módulo é síncrona, operando na borda de subida do `clock_in`. Quando o sinal `reset` está ativo, o contador `count` é zerado e a saída `clock_out` é definida como 0. Em operação normal (quando `reset` não está ativo), o contador `count` é incrementado a cada ciclo de `clock_in`. Quando `count` atinge o valor 26'd50, ele é zerado, e o estado da saída `clock_out` é invertido (`clock_out`). Isso significa que a saída `clock_out` muda de estado a cada 51 ciclos do `clock_in` (contando de 0 a 50). Consequentemente, o período do `clock_out` será 102 ciclos de `clock_in` (51 ciclos para um estado e 51 para o outro), dividindo a frequência de entrada por 102. Este método é uma forma eficaz de gerar um clock de frequência mais baixa e estável a partir de um clock de referência.

4.5.12 Filtro Debouncer

O Módulo Debouncer é um componente fundamental em sistemas digitais que interagem com entradas físicas, como botões ou chaves. A característica inerente a esses dispositivos mecânicos é a ocorrência de "ruído" ou "saltos" (bounces) no sinal elétrico quando o estado é alterado (pressionado ou solto). Sem um debouncer, um único acionamento de um botão poderia ser interpretado como múltiplos acionamentos pelo circuito digital, levando a comportamentos indesejados. A função primordial deste módulo é filtrar esses transientes, garantindo que apenas uma transição de estado estável seja reconhecida.

```
1 module DBounce (
2     input clk,
3
4     input rst,
5
6     input noisy,
7
```

```
8 output reg clean
9 );
10
11 reg [25:0] count;
12 reg new_state;
13
14 always @(posedge clk) begin
15     if (rst) begin
16         count    <= 0;
17         clean    <= 0;
18         new_state <= 0;
19     end else begin
20         if (noisy != new_state) begin
21             new_state <= noisy;
22             count <= 0;
23         end else begin
24             if (count < 26'd50) begin
25                 count <= count + 1;
26             end else begin
27                 clean <= new_state;
28             end
29         end
30     end
31 end
32
33 endmodule
```

Código 4.12 – Código Verilog do Módulo Debouncer

Conforme ilustrado no [Código 4.12](#), o módulo DBounce possui as seguintes entradas e saídas: clk (clock), rst (reset assíncrono), noisy (a entrada com ruído do botão/chave) e clean (a saída estável e "limpa"). Internamente, ele utiliza dois registradores: count, um contador de 26 bits para medir o tempo de estabilização, e new_state, que armazena o estado atual detectado do sinal ruidoso.

A lógica do módulo é síncrona, operando na borda de subida do clock (posedge clk). No reset (rst), todos os registradores são inicializados para zero. Em operação normal, o módulo compara a entrada noisy com o estado new_state. Se houver uma diferença, significa que uma possível transição ocorreu. Nesse caso, new_state é atualizado para o valor de noisy, e o contador count é zerado, iniciando uma nova contagem para estabilização. Se noisy for igual a new_state (indicando que o sinal permaneceu no mesmo estado por pelo menos um ciclo de clock), o contador count é incrementado. Quando count atinge um valor pré-determinado (neste caso, 26'd50), isso indica que o sinal noisy permaneceu estável por um período suficiente, e o valor de new_state é então transferido para a saída clean. Esse atraso controlado garante que apenas transições verdadeiras e estáveis sejam propagadas para a saída, eliminando o efeito de "saltos".

4.5.13 Conversor Binário para BCD

O Conversor Binário para BCD é um componente vital em sistemas digitais, especialmente quando é necessário exibir valores binários em formatos decimais, como em displays de 7 segmentos. Sua função principal é transformar um número binário de entrada em uma série de dígitos BCD (Binary-Coded Decimal), onde cada grupo de 4 bits representa um dígito decimal individual. Este módulo é fundamental para a visualização de dados numéricos que são internamente processados em formato binário, facilitando a interação do usuário com o sistema.

```
1 module BinToBcdConverter (  
2   input  [31:0] binary_in,  
3   output reg [3:0] bcd0,  
4   output reg [3:0] bcd1,  
5   output reg [3:0] bcd2,  
6   output reg [3:0] bcd3,  
7   output reg [3:0] bcd4,  
8   output reg [3:0] bcd5,  
9   output reg [3:0] bcd6,  
10  output reg [3:0] bcd7  
11 );  
12  
13 always @(*) begin  
14     bcd0 = binary_in % 10;  
15     bcd1 = (binary_in / 10) % 10;  
16     bcd2 = (binary_in / 100) % 10;  
17     bcd3 = (binary_in / 1000) % 10;  
18     bcd4 = (binary_in / 10000) % 10;  
19     bcd5 = (binary_in / 100000) % 10;  
20     bcd6 = (binary_in / 1000000) % 10;  
21     bcd7 = (binary_in / 10000000) % 10;  
22 end  
23  
24 endmodule
```

Código 4.13 – Código Verilog do Conversor Binário para BCD

Conforme demonstrado no [Código 4.13](#), o módulo BinToBcdConverter é projetado com uma entrada de 32 bits, **binary_in**, que recebe o valor binário a ser convertido. As saídas consistem em oito registradores de 4 bits, de **bcd0** a **bcd7**, cada um representando um dígito decimal do número original. A lógica interna é implementada dentro de um bloco `always @(*)`, que garante que a saída seja sempre atualizada com base nas mudanças na entrada, caracterizando-o como um circuito puramente combinacional.

O desenvolvimento da lógica baseia-se na aritmética decimal para extrair cada dígito. O dígito menos significativo (**bcd0**) é obtido calculando o resto da divisão de **binary_in** por 10 (operador `% 10`). Para os dígitos subsequentes, o valor de **binary_in** é dividido por uma potência de 10 crescente (10, 100, 1000, etc.) antes de aplicar novamente a operação de módulo 10. Por exemplo, **bcd1** é o resto da divisão de (**binary_in** dividido por 10) por 10, e assim por diante. Esse método permite a decomposição do número binário em seus respectivos dígitos decimais, que podem então ser exibidos individualmente.

4.5.14 Decodificador BCD para 7 Segmentos

O Decodificador BCD para 7 Segmentos é um componente essencial em sistemas digitais que requerem a exibição de informações numéricas em displays de 7 segmentos. Sua função primordial é converter um número binário codificado em decimal (BCD) de 4 bits em um padrão de 7 bits que aciona os segmentos corretos do display para formar o dígito correspondente. Este módulo é crucial para a interface visual do usuário, permitindo que os dados internos do processador sejam facilmente interpretados em plataformas como um FPGA.

```

1 module BcdTo7SegmentDecoder (
2   input [3:0] bcd_in,
3   output reg [6:0] seg_out
4 );
5
6 always @ (*) begin
7   case(bcd_in)
8     4'b0000 : seg_out = 7'b1000000;
9     4'b0001 : seg_out = 7'b1111001;
10    4'b0010 : seg_out = 7'b0100100;
11    4'b0011 : seg_out = 7'b0110000;
12    4'b0100 : seg_out = 7'b0011001;
13    4'b0101 : seg_out = 7'b0010010;
14    4'b0110 : seg_out = 7'b0000010;
15    4'b0111 : seg_out = 7'b1111000;
16    4'b1000 : seg_out = 7'b0000000;
17    4'b1001 : seg_out = 7'b0011000;
18    default: seg_out = 7'b1111111; // Desliga tudo
19  endcase
20 end
21
22 endmodule

```

Código 4.14 – Código Verilog do Decodificador BCD para 7 Segmentos

Conforme demonstrado no [Código 4.14](#), o módulo BcdTo7SegmentDecoder é projetado com uma entrada de 4 bits, `bcd_in`, que recebe o valor BCD (Binary-Coded Decimal) a ser decodificado, e uma saída de 7 bits, `seg_out`, que controla os segmentos do display. A lógica interna é implementada utilizando uma estrutura `case`, que permite mapear cada possível valor de `bcd_in` (de 0 a 9) para o padrão binário correspondente necessário para acender os segmentos corretos do display de 7 segmentos. Por exemplo, quando `bcd_in` é `4'b0000` (representando o dígito 0), a saída `seg_out` é definida como `7'b1000000`, que acende os segmentos apropriados para formar o numeral "0". Para qualquer entrada BCD inválida (ou seja, valores de 10 a 15), a saída `seg_out` é configurada para `7'b1111111`, o que tipicamente resulta em todos os segmentos do display estando apagados ou em um padrão de erro. Este módulo é puramente combinacional, garantindo que a saída seja atualizada instantaneamente com base na entrada, o que é fundamental para a exibição dinâmica e em tempo real de dados.

4.5.15 Módulo de Entrada/Saída (I/O)

O projeto do Módulo de Entrada/Saída (I/O), é fundamental para a interação entre o sistema digital e o usuário. Sua principal função é gerenciar o fluxo de dados de diferentes fontes, como chaves físicas (switches) ou dados externos, e exibir o estado interno do sistema em um conjunto de displays de 7 segmentos. Este módulo atua como uma interface de hardware, selecionando a origem dos dados a serem processados pela lógica principal e, ao mesmo tempo, fornecendo feedback visual do resultado no FPGA.

```

1 module ModuloEntradaSaida (
2     input [9:0] switch_dado,
3     input [31:0] entrada_dado,
4     input IOE,
5     input IOsel,
6     output reg [31:0] saida_dado,
7
8     output [6:0] HEX0,
9     output [6:0] HEX1,
10    output [6:0] HEX2,
11    output [6:0] HEX3,
12    output [6:0] HEX4,
13    output [6:0] HEX5,
14    output [6:0] HEX6,
15    output [6:0] HEX7
16 );
17
18 // Sinais BCD individuais
19 wire [3:0] bcd0, bcd1, bcd2, bcd3;
20 wire [3:0] bcd4, bcd5, bcd6, bcd7;
21
22 // Valor que vai ser exibido
23 reg [31:0] reg_out;
24
25 initial begin
26     reg_out = 32'b0;
27 end
28
29 // Logica de selecao: mostra switches ou entrada
30 always @(*) begin
31
32     if (IOE) begin
33         if (IOsel) begin
34             saida_dado = switch_dado + 32'b0; // concatena zeros nos bits mais altos
35         end
36         else begin
37             reg_out = entrada_dado;
38         end
39     end
40     else begin
41         reg_out = reg_out;
42     end
43 end
44
45 // Conversor Binario -> BCD
46 BinToBcdConverter b2b_inst (
47     .binary_in(reg_out),
48     .bcd0(bcd0),

```

```

49     .bcd1(bcd1),
50     .bcd2(bcd2),
51     .bcd3(bcd3),
52     .bcd4(bcd4),
53     .bcd5(bcd5),
54     .bcd6(bcd6),
55     .bcd7(bcd7)
56 );
57
58 // Decodificadores
59 BcdTo7SegmentDecoder dec0_inst (.bcd_in(bcd0), .seg_out(HEX0));
60 BcdTo7SegmentDecoder dec1_inst (.bcd_in(bcd1), .seg_out(HEX1));
61 BcdTo7SegmentDecoder dec2_inst (.bcd_in(bcd2), .seg_out(HEX2));
62 BcdTo7SegmentDecoder dec3_inst (.bcd_in(bcd3), .seg_out(HEX3));
63 BcdTo7SegmentDecoder dec4_inst (.bcd_in(bcd4), .seg_out(HEX4));
64 BcdTo7SegmentDecoder dec5_inst (.bcd_in(bcd5), .seg_out(HEX5));
65 BcdTo7SegmentDecoder dec6_inst (.bcd_in(bcd6), .seg_out(HEX6));
66 BcdTo7SegmentDecoder dec7_inst (.bcd_in(bcd7), .seg_out(HEX7));
67
68 endmodule

```

Código 4.15 – Código Verilog do Módulo de Entrada/Saída (I/O)

Conforme ilustrado no [Código 4.15](#), o módulo possui diversas entradas e saídas. As entradas `switch_dado`, de 10 bits, são lidas diretamente das chaves DIP do hardware, enquanto a entrada `entrada_dado`, de 32 bits, recebe dados de outro módulo. O sinal `IOE` (Input/Output Enable) controla o fluxo de dados para a saída. Se `IOE` for verdadeiro (1) e `IOsel` também for verdadeiro (1), a saída `saida_dado` recebe o valor de `switch_dado`. Se `IOE` for verdadeiro (1) e `IOsel` for falso (0), o valor de `entrada_dado` é carregado no registrador interno `reg_out` quando o sinal `confirm` for ativado. A lógica de exibição é puramente combinacional: o valor armazenado em `reg_out` é passado para uma instância de `BinToBcdConverter`, que o converte para oito dígitos BCD. Esses dígitos são então processados por decodificadores individuais de 7 segmentos (`BcdTo7SegmentDecoder`), gerando os sinais de controle para os displays `HEX0` a `HEX7`. Esse arranjo permite que o módulo exiba qualquer valor de 32 bits, fornecendo um feedback visual crucial do estado interno do sistema.

4.5.16 Integração Final dos Módulos

A partir do caminho de dados na [Figura 6](#), o código em [Código 4.16](#) demonstra a integração completa de um processador, conectando todos os módulos para formar um sistema funcional. A arquitetura se baseia em um fluxo de dados e controle bem definido, coordenados pela `UnidadeControle`.

```

1 module Processador_Jonas (clk, switches, confirm, reset, HEX0, HEX1, HEX2, HEX3, HEX4,
2   HEX5, HEX6, HEX7);
3     input clk;
4     input reset;
5     input [9:0] switches;

```

```

6      input confirm;
7
8
9      wire SZ;
10     wire JAL;
11     wire JR;
12     wire HLT;
13     wire Jump;
14     wire Branch;
15     wire RegWrite;
16     wire RSsel;
17     wire RTsel;
18     wire DadoSel;
19     wire ResSel;
20     wire PilhaE;
21     wire PilhaOP;
22     wire ALUsrc;
23     wire MemToReg;
24     wire MemRead;
25     wire MemWrite;
26     wire IOE;
27     wire IOsel;
28     wire stall;
29     wire [1:0] IMsel;
30     wire [3:0] ALUOp;
31
32     wire [31:0] instrucao;
33     wire [31:0] dado1;
34     wire [31:0] dado2;
35     wire [31:0] addr_rs;
36     wire [31:0] addr_rt;
37     wire [31:0] addr_rd;
38     wire [31:0] Imediato;
39     wire [31:0] in1;
40     wire [31:0] in2;
41     wire [31:0] resultado;
42     wire [31:0] escrita_dado;
43     wire [31:0] rp;
44     wire [31:0] in_pc;
45     wire [31:0] out_pc;
46     wire [31:0] rp_mem;
47     wire [31:0] mem_out;
48     wire [31:0] addr_leitura;
49     wire [31:0] result_out;
50     wire [31:0] mem_reg_out;
51     wire [31:0] pc_plus_one;
52     wire [31:0] pc_branch;
53     wire [31:0] pc_jump;
54     wire [31:0] pc_jr_jump;
55     wire [31:0] mem_io_out;
56     wire [31:0] saida_io;
57     wire Zero;
58     wire and_branch;
59     wire confirm_limpo;
60     wire ok;
61
62     output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7;
63
64     wire clock;
65

```



```
66     assign ok = ~confirm;
67
68     DBounce db (
69         .clk(clk),
70         .rst(reset),
71         .noisy(ok),
72         .clean(confirm_limpo)
73     );
74
75
76     Divisor_Freq div (
77         .clock_in(clk),
78         .reset(reset),
79         .clock_out(clock)
80     );
81
82     Adder somador_1 (
83         .a(32'd1),
84         .b(out_pc),
85         .out(pc_plus_one)
86     );
87
88     Adder somador_2 (
89         .a(pc_plus_one),
90         .b(Imediato),
91         .out(pc_branch)
92     );
93
94     mux mux_jr (
95         .a(Imediato),
96         .b(dado1),
97         .s(JR),
98         .out(pc_jr_jump)
99     );
100
101     mux mux_jump (
102         .a(pc_plus_one),
103         .b(pc_jr_jump),
104         .s(Jump),
105         .out(pc_jump)
106     );
107
108     assign and_branch = Zero & Branch;
109
110     mux mux_branch (
111         .a(pc_jump),
112         .b(pc_branch),
113         .s(and_branch),
114         .out(in_pc)
115     );
116
117     PC pc (
118         .clock(clock),
119         .reset(reset),
120         .HLT(HLT),
121         .stall(stall),
122         .confirm(confirm_limpo),
123         .in_pc(in_pc),
124         .out_pc(out_pc)
125     );
```

```

126
127     MemoriaInstrucoes mem_instr (
128         .addr_pc(out_pc[5:0]),
129         .q(instrucao)
130     );
131
132     UnidadeControle UC (
133         .opcode(instrucao[31:26]),
134         .JAL(JAL),
135         .JR(JR),
136         .HLT(HLT),
137         .DadoSel(DadoSel),
138         .PilhaE(PilhaE),
139         .PilhaOP(PilhaOP),
140         .SZ(SZ),
141         .ResSel(ResSel),
142         .ALUOp(ALUOp),
143         .MemToReg(MemToReg),
144         .RegWrite(RegWrite),
145         .ALUsrc(ALUsrc),
146         .MemRead(MemRead),
147         .MemWrite(MemWrite),
148         .Branch(Branch),
149         .RSsel(RSsel),
150         .RTsel(RTsel),
151         .IMsel(IMsel),
152         .Jump(Jump),
153         .IOE(IOE),
154         .IOsel(IOsel),
155         .stall(stall)
156     );
157
158     mux mux_rs (
159         .a(instrucao[19:14]),
160         .b(instrucao[25:20]),
161         .s(RSsel),
162         .out(addr_rs)
163     );
164
165     mux mux_rt (
166         .a(instrucao[13:8]),
167         .b(instrucao[19:14]),
168         .s(RTsel),
169         .out(addr_rt)
170     );
171
172     assign addr_rd = instrucao[25:20];
173
174     mux mux_reg_ra (
175         .a(mem_io_out),
176         .b(pc_plus_one), // $ra
177         .s(JAL),
178         .out(escrita_dado)
179     );
180
181     BancoRegistradores banco (
182         .clock(clock),
183         .addr_rs(addr_rs),
184         .addr_rt(addr_rt),
185         .addr_rd(addr_rd),

```

```

186         .escrita_dado(escrita_dado),
187         .rp(rp),
188         .RegWrite(RegWrite),
189         .PilhaE(PilhaE),
190         .JAL(JAL),
191         .dado1(dado1),
192         .dado2(dado2)
193     );
194
195     Extensor extensor (
196         .in1(instrucao[13:0]),
197         .in2(instrucao[19:0]),
198         .in3(instrucao[25:0]),
199         .IMsel(IMsel),
200         .out(Imediato)
201     );
202
203     mux mux_sel_in1_ULA (
204         .a(dado1),
205         .b(dado2),
206         .s(DadoSel),
207         .out(in1)
208     );
209
210     mux mux_sel_in2_ULA (
211         .a(dado2),
212         .b(Imediato),
213         .s(ALUsrc),
214         .out(in2)
215     );
216
217     ULA ula (
218         .in1(in1),
219         .in2(in2),
220         .ALUOp(ALUOp),
221         .sz(SZ),
222         .result(resultado),
223         .branch(Zero)
224     );
225
226     UC_Pilha pilha_ctrl (
227         .rp_in(dado2),
228         .PilhaE(PilhaE),
229         .PilhaOP(PilhaOP),
230         .rp_out(rp),
231         .rp_mem(rp_mem)
232     );
233
234     mux mux_addr_leitura (
235         .a(resultado),
236         .b(rp_mem),
237         .s(PilhaE),
238         .out(addr_leitura)
239     );
240
241     MemoriaDados mem_dados (
242         .clk(clock),
243         .data(in1),
244         .addr(addr_leitura[5:0]),
245         .MemWrite(MemWrite),

```

```

246         .MemRead(MemRead),
247         .q(mem_out)
248     );
249
250     mux mux_result (
251         .a(resultado),
252         .b(Imediato),
253         .s(ResSel),
254         .out(result_out)
255     );
256
257     mux mux_MemReg (
258         .a(result_out),
259         .b(mem_out),
260         .s(MemToReg),
261         .out(mem_reg_out)
262     );
263
264     mux mux_io (
265         .a(mem_reg_out),
266         .b(saida_io),
267         .s(IOsel),
268         .out(mem_io_out)
269     );
270
271
272     ModuloEntradaSaida in_out (
273         .switch_dado(switches),
274         .entrada_dado(in1),
275         .IOE(IOE),
276         .IOsel(IOsel),
277         .saida_dado(saida_io),
278         .HEX0(HEX0),
279         .HEX1(HEX1),
280         .HEX2(HEX2),
281         .HEX3(HEX3),
282         .HEX4(HEX4),
283         .HEX5(HEX5),
284         .HEX6(HEX6),
285         .HEX7(HEX7)
286     );
287
288 endmodule

```

Código 4.16 – Código Verilog da Integração Final com todos os Módulos

O coração do controle de fluxo é o módulo PC (Program Counter), que armazena o endereço da instrução atual. Ele é incrementado a cada ciclo pelo `somador_1` para avançar para a próxima instrução. Para lidar com desvios condicionais e incondicionais, vários multiplexadores e somadores são usados. O `mux_jr` e o `mux_jump` gerenciam desvios para endereços de retorno ou saltos diretos. O `somador_2` calcula o endereço de desvio (branch) ao somar o endereço do PC com o valor imediato estendido. Por fim, o `mux_branch` seleciona o próximo endereço do PC com base na condição de desvio (`and_branch`), que é a combinação do flag de zero gerado pela ULA e o sinal de controle Branch. O endereço final é enviado para a `MemoriaInstrucoes` para buscar a próxima instrução.

A instrução lida da memória é enviada para a **UnidadeControle**, que decodifica o **opcode** e gera todos os sinais de controle necessários para o restante do processador. O módulo **Extensor** é responsável por expandir os valores imediatos da instrução, que podem ser usados como operandos para a ULA ou para o cálculo de endereços. A seleção dos operandos para a ULA é feita por dois multiplexadores: o **mux_sel_in1_ULA** escolhe entre **dado1** e **dado2** e o **mux_sel_in2_ULA** seleciona entre **dado2** e o valor imediato, adaptando o caminho de dados para diferentes tipos de instrução. A ULA executa a operação especificada pelo sinal **ALUOp** nos operandos selecionados, gerando o resultado (**resultado**) e o flag de zero (**Zero**), que é usado, junto com o sinal de controle **Branch**, para determinar se um desvio condicional deve ser tomado.

O **BancoRegistadores** é o principal ponto de armazenamento de dados, lendo os operandos **dado1** e **dado2** e recebendo o resultado da operação no sinal **escrita_dado**. Esse sinal é alimentado por uma cadeia de multiplexadores (**mux_result**, **mux_MemReg** e **mux_io**) que permite que o resultado da ULA, o dado lido da **MemoriaDados** ou os dados de I/O sejam escritos de volta nos registradores. A **MemoriaDados** é acessada pelo endereço **addr_leitura**, que é selecionado entre o resultado da ULA e o ponteiro da pilha.

O sistema também inclui módulos para funcionalidades específicas. A **UC_Pilha** gerencia o ponteiro da pilha, permitindo operações de push e pop. O **ModuloEntradaSaida** gerencia a interação com o usuário através de chaves (**switches**) e displays de sete segmentos (**HEX**). Por fim, os módulos **DBounce** e **Divisor_Freq** garantem a estabilidade do sistema, filtrando o ruído de um botão de confirmação e gerando um clock de frequência mais baixa para a lógica do processador.

5 Resultados Obtidos e Discussões

Esta parte do projeto foi detalhado o processo de implementação e validação dos componentes fundamentais do processador. A metodologia adotada nesta fase do projeto consistiu no desenvolvimento dos módulos individualmente em Verilog HDL, seguido pela verificação funcional individual de cada bloco, uma abordagem essencial para garantir a corretude e a robustez do sistema final.

Foram implementados todos os módulos que compõem o caminho de dados: a Unidade Lógica e Aritmética (ULA), o Banco de Registradores, o Contador de Programa (PC) e as Memórias de Dados e de Instruções, etc. Para cada módulo, foram realizados testes de simulação com a *Waveform* e a análise dessas formas de onda geradas permitiu verificar a conformidade da lógica implementada com o comportamento esperado, além de possibilitar a depuração de possíveis erros. Os resultados apresentados a seguir demonstram o funcionamento de cada componente isoladamente, estabelecendo uma base sólida para a futura etapa de integração do processador.

5.1 Formas de Onda

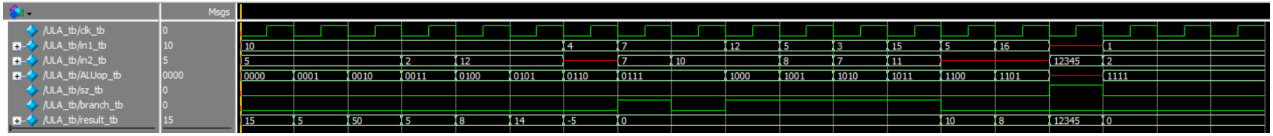
Esta seção é dedicada à apresentação e análise das formas de onda obtidas através da simulação de cada módulo fundamental do processador. Com o objetivo de simplificar a visualização, os módulos que não possuem forma de onda nessa seção foram omitidos, mas seu funcionamento foi garantido a partir de testes no kit FPGA.

Através delas, é possível observar em detalhe a relação temporal entre os sinais de entrada, os sinais de controle e as saídas resultantes. A análise ciclo a ciclo permite validar a lógica síncrona, as operações combinacionais e a resposta geral do sistema às entradas aplicadas durante os testes. As figuras e tabelas a seguir documentam a verificação de cada componente chave, servindo como comprovação gráfica do funcionamento adequado do processador.

5.1.1 Teste da Unidade Lógica e Aritmética

Os testes realizados na Unidade Lógica e Aritmética (ULA) foram projetados para validar sistematicamente todas as suas funcionalidades, abrangendo desde operações aritméticas e lógicas básicas até instruções de desvio condicional e casos de controle especiais. A análise dos 17 casos de teste, demonstra o comportamento correto e esperado do módulo em todas as situações especificadas.

Figura 11 – Waveform para as entradas e saídas da ULA para todas as operações da Tabela 5



Fonte: O Autor

A forma de onda da simulação, Figura 11, demonstra a validação funcional da Unidade Lógica e Aritmética (ULA) através de uma sequência de 17 ciclos de clock. Cada ciclo testa uma operação distinta ou um caso de controle específico, confirmando que as saídas **result** e **branch** correspondem aos valores esperados para cada combinação de entradas. A tabela a seguir, Tabela 7, detalha os valores de cada sinal em cada ciclo de clock, conforme feito na simulação e apresenta os sinais de saída.

Tabela 7 – Resultados dos Casos de Teste da ULA

Ciclo	Operação	in1	in2	ALUop	sz	result	branch
1	add	10	5	0000	0	15	0
2	sub	10	5	0001	0	5	0
3	mult	10	5	0010	0	50	0
4	div	10	2	0011	0	5	0
5	AND	...1010	...1100	0100	0	...1000	0
6	OR	...1010	...1100	0101	0	...1110	0
7	NOT	...0100	N/A	0110	0	...1011	0
8	beq	7	7	0111	0	0	1
9	beq	7	10	0111	0	0	0
10	bge	12	10	1000	0	0	1
11	ble	5	8	1001	0	0	1
12	blt	3	7	1010	0	0	1
13	bgt	15	11	1011	0	0	1
14	sl	5	N/A	1100	0	10	0
15	sr	16	N/A	1101	0	8	0
16	Load/Store	N/A	12345	N/A	1	12345	0
17	Default	1	2	1111	0	0	0

As operações aritméticas (adição, subtração, multiplicação e divisão) e lógicas (AND, OR e NOT) foram validadas com sucesso. Conforme os casos de teste 1 a 7, a

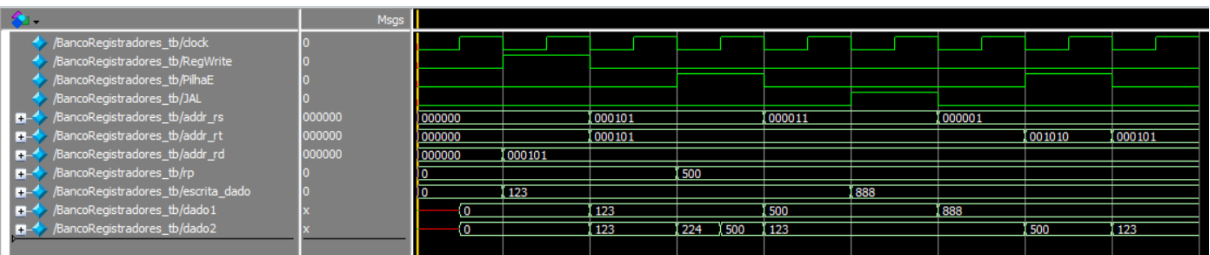
saída **result** correspondeu precisamente ao resultado matemático ou lógico esperado para os valores de entrada, enquanto a saída **branch** permaneceu corretamente inativa (em nível lógico 0). De maneira similar, os testes de deslocamento de bits (casos 14 e 15) confirmaram que as operações *shift left* e *shift right* produziram os resultados corretos, deslocando os bits da entrada **in1** adequadamente.

Para as instruções de desvio condicional (casos 8 a 13), o foco da validação foi a saída **branch**. Em todos os testes, incluindo BEQ (desvio se igual), BGE (maior ou igual), BLE (menor ou igual), BLT (menor que) e BGT (maior que), a saída **branch** foi corretamente ativada (1) quando a condição era verdadeira e desativada (0) quando era falsa. Conforme o projeto da ULA, a saída **result** foi mantida em zero durante essas comparações. Por fim, os casos de controle especiais (16 e 17) também foram validados: com **sz=1**, a ULA atuou corretamente em modo "bypass", quando é feito instruções de Load e Store para endereçamento direto, fazendo com que a saída **result** espelhasse a entrada **in2**; para uma operação inválida (**ALUop=1111**), o módulo retornou os valores padrão de zero, confirmando a robustez da implementação.

5.1.2 Teste do Banco de Registradores

Os testes para o módulo Banco de Registradores foram elaborados para validar de forma abrangente suas múltiplas funcionalidades. O foco foi verificar a correção das operações de escrita síncrona, tanto as de propósito geral (via **RegWrite**) quanto as especializadas (via **PilhaE** e **JAL**). Além disso, foram testadas as operações de leitura combinacional, incluindo a lógica condicional da porta de saída **dado2**.

Figura 12 – Forma de onda da simulação para o módulo Banco de Registradores.



Fonte: O Autor

A forma de onda da simulação, [Figura 12](#), demonstra a validação funcional do Banco de Registradores através de uma sequência de operações de escrita e leitura. Cada ciclo de clock testa uma funcionalidade chave, confirmando que os dados são escritos corretamente e que as saídas **dado1** e **dado2** refletem os valores esperados. A tabela a seguir, [Tabela 8](#), detalha os valores dos sinais em cada etapa da simulação.

Tabela 8 – Resultados dos Casos de Teste do Banco de Registradores

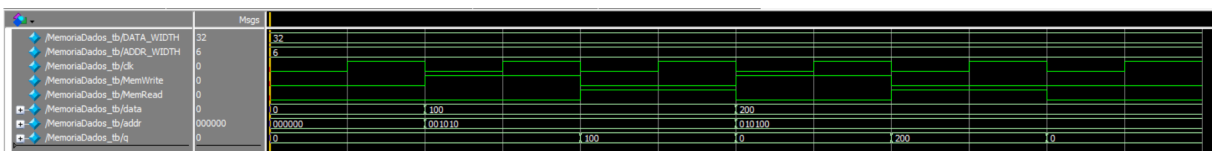
Ciclo	Operação	RegWrite	PE	JAL	Endereços	Entrada	dado1	dado2
1	Escrita	1	0	0	rd=5	dado=123	0	0
2	Leitura	0	0	0	rs=5, rt=5	N/A	123	123
3	Escrita \$rp	0	1	0	N/A	rp=500	123	224
4	Leitura \$rp	0	0	0	rs=3	N/A	500	123
5	Escrita \$ra	0	1	1	N/A	dado=888	500	500
6	Leitura \$ra	0	0	0	rs=1	N/A	888	123
7	Leitura \$rp	0	1	0	rt=10	N/A	888	500
8	Leitura	0	0	0	rt=5	N/A	888	123

A análise da simulação confirma o robusto funcionamento do Banco de Registradores. O ciclo 1 demonstra uma escrita geral bem-sucedida com **RegWrite**, armazenando o valor 123 no registrador 5, o que é validado no ciclo 2 pela leitura correta em ambas as saídas. Os ciclos subsequentes validam as escritas especializadas: no ciclo 3, a ativação de **PilhaE** escreve com sucesso o valor 500 no registrador de pilha (*rp*, endereço 3), e no ciclo 5, o sinal **JAL** escreve 888 no registrador de endereço de retorno (*ra*, endereço 1), com ambas as escritas sendo confirmadas por leituras nos ciclos 4 e 6. Finalmente, os ciclos 7 e 8 validam a lógica condicional da saída **dado2**, que corretamente seleciona sua fonte entre o registrador *rp* (quando **PilhaE**=1) e o registrador apontado por **addr_rt** (quando **PilhaE**=0).

5.1.3 Teste da Memória RAM

Os testes realizados na Memória de Dados foram projetados para validar sistematicamente suas funcionalidades essenciais, abrangendo a escrita síncrona de dados em endereços específicos, a leitura combinacional desses dados e o comportamento da saída quando a leitura está desabilitada. A análise da simulação demonstra o comportamento correto e esperado do módulo.

Figura 13 – Forma de onda da simulação para o módulo MemoriaDados.



Fonte: O Autor

A forma de onda da simulação, [Figura 13](#), demonstra a validação funcional da Memória de Dados através de uma sequência de operações. Cada etapa testa uma funcionalidade chave, confirmando que a saída `q` corresponde aos valores esperados para cada combinação de sinais de controle e endereços. A tabela a seguir, [Tabela 9](#), detalha os valores de cada sinal em cada ciclo da simulação.

Tabela 9 – Resultados dos Casos de Teste da Memória de Dados

Ciclo	Operação	MemWrite	MemRead	addr	data	q
1	Escrita R10	1	0	10	100	0
2	Leitura R10	0	1	10	N/A	100
3	Escrita R20	1	0	20	200	0
4	Leitura R20	0	1	20	N/A	200
5	Desativado	0	0	10	N/A	0

A análise dos resultados da simulação confirma o funcionamento correto da memória. No ciclo 1, o dado 100 é escrito no endereço 10 na borda de subida do clock, enquanto **MemWrite** está ativo. O ciclo 2 valida o sucesso dessa escrita ao ler o mesmo endereço com **MemRead** ativo, fazendo com que a saída **q** apresente o valor 100. De forma análoga, os ciclos 3 e 4 demonstram a escrita e leitura bem-sucedidas do dado 200 no endereço 20. Por fim, o ciclo 5 confirma que, quando a leitura é desabilitada (**MemRead**=0), a saída **q** assume o valor padrão de zero, conforme especificado no projeto.

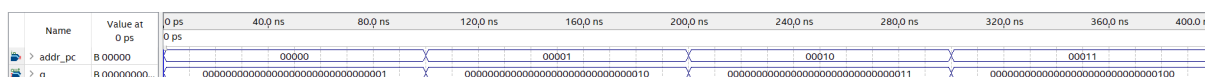
5.1.4 Teste da Memória ROM

O teste realizado na Memória de Instruções visa validar sua função primária como uma ROM (Memória Somente de Leitura): fornecer de forma correta e imediata a instrução correspondente ao endereço de entrada fornecido pelo Contador de Programa (PC). A análise da simulação demonstra que o módulo se comporta como uma memória combinacional, respondendo instantaneamente às mudanças de endereço.

[illegible]

Código 5.1 – Instruções a serem executadas no arquivo `codigo teste.txt`

Figura 14 – Forma de onda da simulação para o módulo MemoriaInstrucoes.



Fonte: O Autor

A forma de onda da simulação, [Figura 15](#), demonstra a validação funcional do PC. É possível observar que a saída `out_pc` só é atualizada nas bordas de subida do sinal `clock` e apenas quando o sinal `HLT` está desativado. A tabela a seguir, [Tabela 11](#), detalha os valores dos sinais nos momentos mais relevantes da simulação, que são as bordas de subida do clock.

Tabela 11 – Resultados dos Casos de Teste do PC por
Borda de Clock

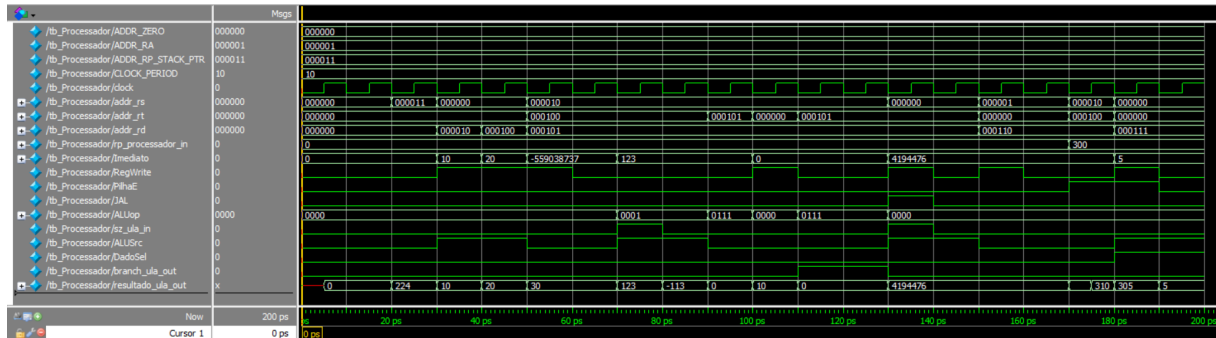
Borda de Subida do Clock	Operação	HLT	in_pc	out_pc
1	$PC \leftarrow PC + 1$	0	0	0
2	$PC \leftarrow PC + 1$	0	1	1
3	$PC \leftarrow PC + 1$	0	2	2
4	$PC \leftarrow PC + 1$	0	3	3
5	$PC \leftarrow PC + 1$	0	4	4
6	PC em "Halt"(mantém)	1	5	4
7	PC em "Halt"(mantém)	1	6	4

A análise dos resultados da simulação confirma o funcionamento correto do Contador de Programa. Durante os cinco primeiros ciclos de clock, com o sinal `HLT` em nível baixo, o PC atualiza seu valor com sucesso na borda de subida de cada pulso, carregando sequencialmente os valores de 0 a 4 da entrada `in_pc`. A partir de aproximadamente 480 ns, o sinal `HLT` é ativado. Conforme observado na sexta e sétima bordas de subida do clock, mesmo com a entrada `in_pc` mudando, a saída `out_pc` permanece fixa em 4, validando a lógica de "halt" que impede a atualização do registrador.

5.1.6 Teste Integração Banco de Registradores e ULA

Os testes de integração do módulo **Processador** foram projetados para validar o caminho de dados completo, verificando a correta interação entre o Banco de Registradores, os Multiplexadores e a ULA. O objetivo é confirmar que os dados fluem corretamente desde a leitura dos operandos até a escrita do resultado, sob a orquestração dos diversos sinais de controle. A análise da simulação demonstra que, após a correção da conexão do clock para o Banco de Registradores, todos os cenários de teste foram executados com sucesso.

Figura 16 – Forma de onda da simulação do módulo Processador integrado.



Fonte: O Autor

A forma de onda da simulação, [Figura 16](#), demonstra a validação funcional do caminho de dados. Cada cenário de teste valida uma funcionalidade específica, como operações tipo-R, tipo-I, desvios e escritas especializadas, confirmando que as saídas **resultado_ula_out** e **branch_ula_out** correspondem aos valores esperados. A tabela a seguir, [Tabela 12](#), resume os principais cenários de teste executados e seus respectivos resultados.

Tabela 12 – Resultados dos Testes do Processador por Etapa da Simulação

Tempo (ns)	Operação	Resultado	Branch
30-40	Escrita de valor imediato (10) no registrador \$2.	10	0
40-50	Escrita de valor imediato (20) no registrador \$4.	20	0
50-60	Soma tipo-R: \$5 = \$2 + \$4.	30	0
70-80	Teste de Bypass da ULA com sz=1.	123	0
90-100	Desvio se Igual (BEQ) com operandos diferentes (10 e 30).	0	0 (Falso)
110-120	Desvio se Igual (BEQ) com operandos iguais (10 e 10).	0	1 (Verdadeiro)
130-160	Salva o endereço de retorno 0x4000AC em \$ra e valida a leitura posterior.	0x4000AC	0
170-180	Escrita do valor 300 no registrador de pilha \$rp.	N/A	N/A
180-190	Leitura de \$rp (300) via dado2 e soma com imediato.	305	0

A execução bem-sucedida de todos os cenários de teste valida a correta integração do caminho de dados do processador. As operações do tipo-I e tipo-R demonstram que os multiplexadores selecionam adequadamente entre valores de registradores e imediatos. A lógica de desvio funciona como esperado, ativando o sinal **branch** corretamente. Mais

importante, as escritas e leituras especializadas controladas por JAL e PilhaE confirmam que os registradores de função específica (*ra* e *rp*) são atualizados e lidos com sucesso.

5.2 Teste no *kit* FPGA

Para verificar o funcionamento do processador foi efetuado dois testes principais que implementam as principais instruções do processador como os saltos condicionais, incondicionais, uso da memória e também de instruções do tipo R e I.

5.2.1 Teste Fibonacci

O algoritmo para calcular o **número de Fibonacci** para uma posição específica é demonstrado no **Código Mnemônico Fibonacci** (Código 5.2). Primeiramente, um valor de entrada, que representa a posição desejada do número de Fibonacci, é inserido. O algoritmo considera a primeira posição como 1; assim, se o valor de entrada for 1 (nas chaves SW0 até SW9), ele corresponderá à segunda posição do número de Fibonacci.

O valor de entrada é então salvo - após apertar o botão de confirmar no KEY3 do FPGA - na **posição de memória 2** e, subsequentemente, carregado no **registrador R4**. A partir desse ponto, um *loop* é executado, calculando cada valor de Fibonacci até que a posição final solicitada seja alcançada. Isso satisfaz a condição de *branch* e encerra o programa, exibindo o último valor encontrado.

O **Código de Máquina Fibonacci** (Código 5.3) representa as instruções em formato binário, seguindo os padrões de formato e *opcode* estabelecidos no desenvolvimento e planejamento do processador.

As figuras **Teste Fibonacci entrada 1** (Figura 17), **Teste Fibonacci entrada 10** (Figura 18), e **Teste Fibonacci entrada 16** (Figura 19) ilustram o funcionamento do código com diferentes valores de entrada, mostrando os resultados para as posições 1, 10 e 16, respectivamente.

```
1 IN R9
2 sw R9, 2
3 lw R4, 2
4 li R5, 1
5 li R6, 0
6 beq R4, $zero, 6
7 move R7, R5
8 add R5, R5, R6
9 move R6, R7
10 subi R4, R4, 1
11 OUT R5
12 j 5
13 HLT
```

Código 5.2 – Código Mnemônico Fibonacci

```

1 10000000100100000000000000000000
2 01011000100100000000000000000010
3 01010100010000000000000000000010
4 01010000010100000000000000000001
5 01010000011000000000000000000000
6 001111000100000000000000000000110
7 01001100011100010100000000000000
8 00000000010100010100011000000000
9 01001100011000011100000000000000
10 00100000010000010000000000000001
11 10000100010100000000000000000000
12 01101100000000000000000000000101
13 10001100000000000000000000000000

```

Código 5.3 – Instruções a serem executadas no arquivo codigo_teste.txt

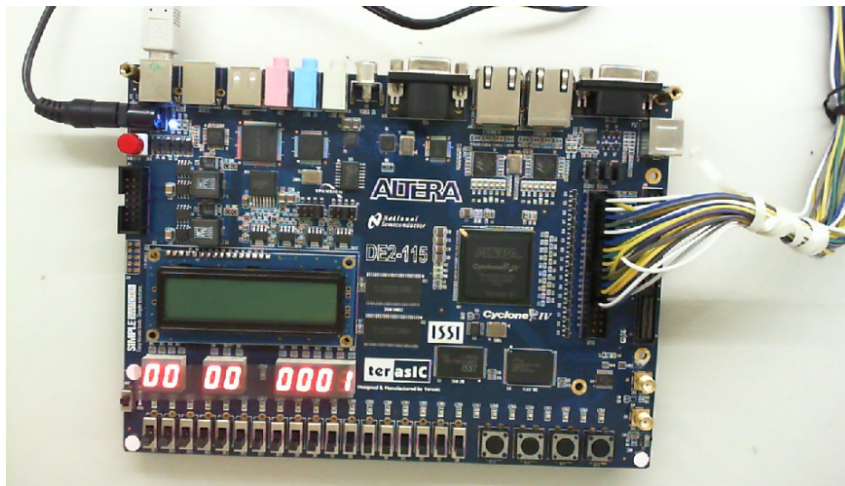


Figura 17 – Teste Fibonacci entrada 1

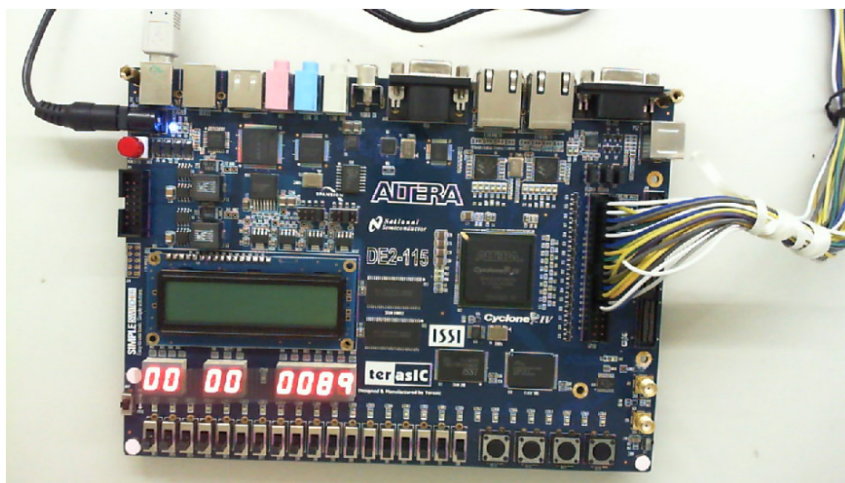


Figura 18 – Teste Fibonacci entrada 10

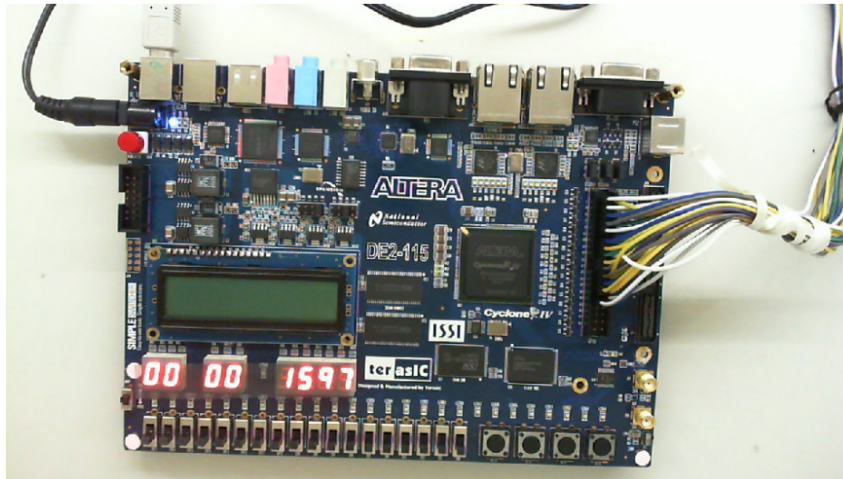


Figura 19 – Teste Fibonacci entrada 16

5.2.2 Teste JR e JAL

Este teste demonstra o funcionamento das instruções de salto **JR** (Jump Register) e **JAL** (Jump And Link) em um contexto prático. O código, apresentado no **Código Mnemônico JR e JAL** ([Código 5.4](#)), inicia lendo um valor de entrada para o registrador R4.

A instrução ‘jal 5’ é então executada. Esta instrução realiza um salto para o endereço de memória 5 e, crucialmente, armazena o endereço da instrução subsequente (aquela que vem logo após o ‘jal’) em um registrador de ligação (neste caso, implicitamente o R1, que atua como registrador de retorno, conforme a notação comum em MIPS como \$ra).

Após o salto, o programa continua a execução a partir do endereço 5. Lá, a instrução ‘add R4, R4, R4’ duplica o valor contido em R4, armazenando o resultado novamente em R4. Em seguida, a instrução ‘jr R1’ (ou ‘jr \$ra’) é responsável por retornar a execução para o endereço que foi salvo pelo ‘jal’.

De volta à sequência original, a instrução ‘add R5, R4, R4’ é executada. Note que, a essa altura, o valor em R4 já foi duplicado pela função para a qual o ‘jal’ saltou. Portanto, esta ‘add’ efetivamente quadruplica o valor original de entrada, armazenando o resultado em R5. Finalmente, o valor de R5 é exibido na saída e o programa é encerrado pela instrução ‘HLT’.

O **Código de Máquina JR e JAL** ([Código 5.5](#)) apresenta a representação binária dessas instruções, aderindo aos formatos e códigos de operação definidos para o processador.

Para ilustrar o comportamento do código com diferentes entradas, observe as figuras: [Figura 20](#) mostra o resultado para a entrada 1, [Figura 21](#) para a entrada 17, e a

(Figura 22) para a entrada 512. Em todos os casos, o valor de saída será o quádruplo do valor de entrada.

```

1 IN R4
2 jal 5
3 add R5, R4, R4
4 OUT R5
5 HLT
6 add R4, R4, R4
7 jr R1 ($ra)

```

Código 5.4 – Código Mnemônico JR e JAL

```

1 1000000001000000000000000000000000
2 0111010000000000000000000000000101
3 0000000001010001000001000000000000
4 1000010001010000000000000000000000
5 1000110000000000000000000000000000
6 0000000001000001000001000000000000
7 0111000000010000000000000000000000

```

Código 5.5 – Instruções a serem executadas no arquivo codigo_teste.txt

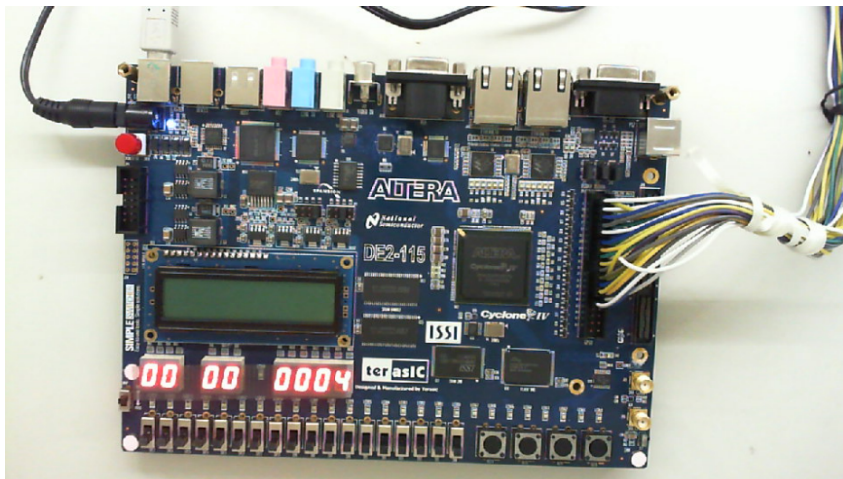


Figura 20 – Teste JR e JAL entrada 1

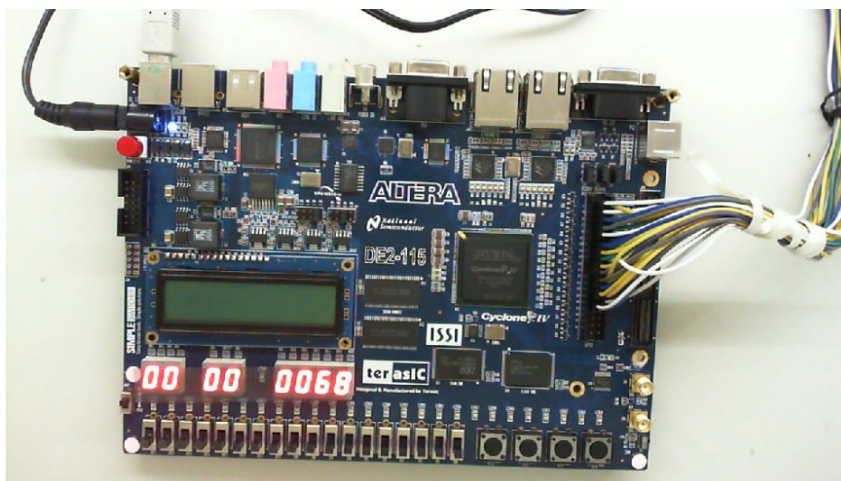


Figura 21 – Teste JR e JAL entrada 17

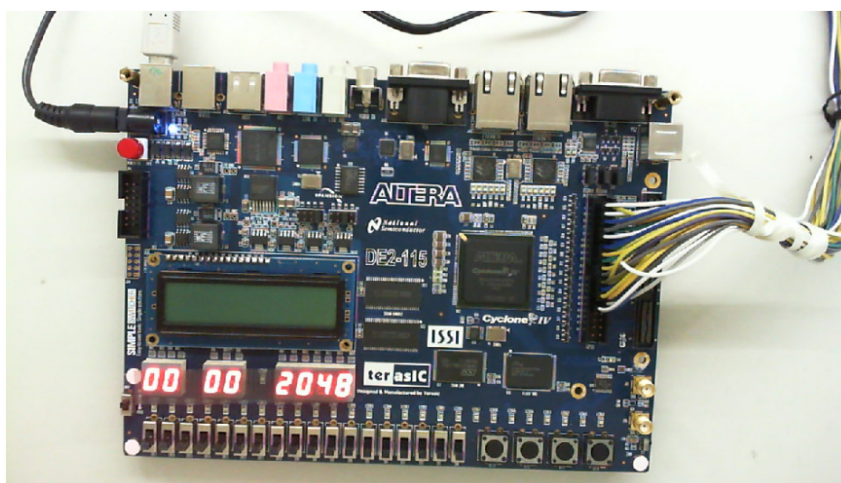


Figura 22 – Teste JR e JAL entrada 512

6 Considerações Finais

Esta etapa final do projeto **consolida com êxito** os conceitos e a arquitetura definidos para o processador. Esse avanço significativo foi alcançado através de duas fases fundamentais. Na **primeira fase**, focamos na **definição da arquitetura** e do **conjunto de instruções**, bem como na especificação dos **módulos essenciais do caminho de dados**. A **segunda fase** compreendeu o desenvolvimento, em **Verilog HDL**, dos principais módulos do processador, como a **ULA (Unidade Lógica e Aritmética)**, o **Banco de Registradores**, o **PC (Program Counter)** e as **Memórias**. Um passo decisivo nessa fase foi a **integração inicial entre a ULA e o Banco de Registradores**. Os **últimos passos** agora foram dedicados à **finalização e integração** dos módulos de controle, memória e ao sistema de controle de entrada e saída de dados.

Os **resultados obtidos validaram o funcionamento correto dos módulos**, estabelecendo uma base **sólida e confiável**. Isso confirma a **viabilidade do design proposto** e permite um avanço seguro para as próximas fases em futuras disciplinas que utilizarão o processador desenvolvido.

Durante o desenvolvimento, surgiram alguns **pontos de dificuldade**. Notavelmente, a realização de **testes para debug dos códigos** foi um desafio quando módulos não operavam como esperado. Houve também mudanças na lógica inicial do DataPath, como a substituição de um *mux* de entrada e saída na entrada do módulo por duas entradas selecionadas internamente. Além disso, existe possíveis melhorias para a otimização das memórias, que apresentaram um tempo de compilação um pouco acima do esperado, otimizá-las complementaria a performance geral do processador.

Referências

- EDITORIAIS DA TMLT. *10 diferenças entre Von Neumann e Harvard architecture*. 2021. Acesso em: 14 maio 2022. Disponível em: <<https://www.currentschoolnews.com/pt/not%C3%A7%C3%A3o/diferen%C3%A7a-entre-a-arquitetura-de-von-neumann-e-harvard/>>. Citado na página 12.
- GERSNOVIEZ, A. et al. UCOMIPSIM 2.0: Pipelined mips architecture simulator. In: *Libro de Actas*. Córdoba, Spain: [s.n.], 2020. {andresgm, mbrox, el1movim, i22suroj, el1momoc}@uco.es. Citado na página 27.
- GIBBONS, A. *MIPS Datapath*. n.d. <<https://mi.eng.cam.ac.uk/~ahg/MIPS-Datapath/>>. Accessed: 2025-05-14. Citado na página 27.
- HEATH, S. *Microprocessor Architectures: RISC, CISC and DSP*. 2nd. ed. [S.l.]: Wiley, 2014. PDF disponível. Citado na página 11.
- HENNESSY, J. L.; PATTERSON, D. A. *Arquitetura de Computadores: uma abordagem quantitativa*. 5th. ed. Rio de Janeiro, Brasil: Elsevier, 2012. Tradução de: Computer architecture: a quantitative approach. ISBN 978-85-352-6122-6. Disponível em: <<https://www.elsevier.com/pt-br>>. Citado 3 vezes nas páginas 11, 12 e 13.
- NELLIS, S. *MIPS shifts strategy toward robots by designing its own chips*. 2025. Accessed: 2025-05-12. Disponível em: <<https://www.reuters.com/technology/artificial-intelligence/mips-shifts-strategy-toward-robots-designing-chips-2025-03-04>>. Citado na página 15.
- (ORG.), E. R. *MIPS RISC Architecture*. 1999. <<https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/mips/index.html>>. Acesso em: maio de 2025. Citado 2 vezes nas páginas 13 e 14.
- PATTERSON, D. A.; HENNESSY, J. L. *Computer architecture: a quantitative approach*. 3. ed. [S.l.]: Morgan Kaufmann, 2006. Citado na página 13.
- SANTOS, H. C. R. dos. *Desenvolvimento de um Sistema Computacional Composto por Processador, Memória e Interface de Comunicação em FPGA*. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) — Universidade Federal de São Paulo (UNIFESP), São José dos Campos, SP, 2014. Orientador: Prof. Dr. Tiago de Oliveira. Citado na página 27.
- STALLINGS, W. *Arquitetura e Organização de Computadores*. 8th. ed. São Paulo, SP, Brasil: Pearson Prentice Hall, 2010. Tradução autorizada do original em inglês: *Computer organization and architecture: designing for performance, eighth edition*. ISBN 978-85-7605-564-8. Citado 8 vezes nas páginas 7, 10, 11, 12, 15, 16, 17 e 18.